FILE: scripts/slime.gd (1036 lines)

```
class_name Slime
extends Node3D
## The jelly slime entity: handles player control, follower breadcrumb AI, or autonomous roamer AI,
## fluid deformation, liquid mode (Z), climb/engulf, splitting (E, Q), and merging (M).

signal poked(local_point: Vector3, impulse: Vector3)

enum Role {
    PLAYER,
    FOLLOWER,
    ROAMER
}

@export var role: Role = Role.PLAYER

@export_group("Nodes")
@export var body: MeshInstance3D
@export var camera: Camera3D
@export var floor_mesh: MeshInstance3D
@export var props: SlimeProps

@export_group("Movement")
@export var walk_speed := 3.0
@export var run_speed := 10.0
@export var liquid_speed := 2.2
@export var velocity_response := 8.0
@export var arena_radius := 190.0

@export_group("Size")
@export var start_scale := 1.0
@export var scale_response := 3.5
@export var floor_y := -1.2
@export var height_per_scale := 1.2
@export var body_radius := 1.6

@export_group("Crawl and lean")
@export var crawl_rate := 2.6
@export var speed_for_full_crawl := 3.0
@export var lean_gain := 0.35
@export var lean_stiffness := 60.0
@export var lean_damping := 7.0
@export var ripple_base := 0.12
@export var ripple_per_speed := 0.45

@export_group("Poke")
@export var poke_strength := 0.55
@export var poke_downward := 0.25

@export_group("Eyes")
@export var facing_response := 8.0
@export var moving_threshold := 0.3
@export var blink_period := 4.0
@export var blink_start := 3.85
@export var blink_closed_amount := 0.12
@export var start_facing := 0.45

@export_group("Liquid and Engulf")
@export var liquid_height_ratio := 0.22
@export var liquid_spread_ratio := 2.13
@export var liquid_clearance_ratio := 0.12
@export var devour_duration := 1.6

@export_group("Follower & Roamer AI")
@export var target_leader: Slime = null
@export var follow_distance := 3.2

var velocity := Vector3.ZERO
var slime_scale := 1.0
var target_scale := 1.0
var speed01 := 0.0

# Vertical motion & ground support
var support_y := -1.2
var vertical_vel := 0.0
var is_on_ground := true
var ground_prop = null

# Liquid mode (Z key)
var liquid_mode := false
var liquid_amt := 0.0

# Engulfing (Z on top of larger object)
var engulf_target = null
var engulf_progress := 0.0

# ---- Fission (budding split) animation state ----
var is_fissioning := false
var fission_timer := 0.0
var fission_duration := 0.38
var fission_fwd := Vector3.FORWARD
var fission_start_scale := 1.0
var fission_target_scale := 0.794

# CLONE budding state
var is_budding := false
var bud_timer := 0.0
var bud_duration := 0.38
var bud_parent: Slime = null
var bud_target_scale := 1.0
var bud_fwd := Vector3.FORWARD

# Merge animation state
var is_merging := false
var merge_target: Slime = null
var merge_timer := 0.0
var merge_duration := 0.45
var merge_source_scale := 1.0
var _has_merged := false

# Physics & deformation state (per-slime)
var calm := 0.0
var touch_time := -10.0
var cur_max := 3.0
var squeeze_k := 1.0
var sq_dir := Vector3(1, 0, 0)
var splat_x := 0.0
var splat_v := 0.0
var spread_x := 0.0
var spread_v := 0.0
var sharp_amt := 0.0
var contact_r := 2.15
var impact_dir := Vector3(0, 0, 1)
var press_amt := 0.0

# Per-slime detected gap state
var cur_gap_center := Vector2.ZERO
var cur_gap_w := INF
var cur_gap_dir := Vector2.ZERO

# Breadcrumb history for followers to trace
var breadcrumbs: Array[Dictionary] = []
var _breadcrumb_timer := 0.0

# Internal props being digested
var inside_props: Array = []

# AI internal state
var _ai_target_pos := Vector3.ZERO
var _ai_roam_timer := 0.0
var _ai_recovery_timer := 0.0
var _ai_target_prop = null
var _ai_stuck_timer := 0.0
var _ai_last_pos := Vector3.ZERO
# Follower-specific: flatten-to-fit flag (suppresses engulf while navigating low areas)
var _following_flatten := false
var _follower_stuck_timer := 0.0

var _mat: ShaderMaterial
var _time := 0.0
var _impulse_time := 10.0
var _impulse2_time := 10.0
var _crawl_dir := Vector3(0, 0, 1)
var _crawl_phase := 0.0
var _facing_yaw := 0.0
var _lean := Vector3.ZERO
var _lean_vel := Vector3.ZERO

var time: float:
    get:
        return _time


func get_volume() -> float:
    ## Authoritative volume of the slime based on target_scale
    return (4.0 / 3.0) * PI * pow(body_radius * target_scale, 3.0)


func get_visual_volume() -> float:
    return (4.0 / 3.0) * PI * pow(body_radius * slime_scale, 3.0)



func _ready() -> void:
    if not is_budding:
        slime_scale = start_scale
        target_scale = start_scale
        _facing_yaw = start_facing
        support_y = floor_y

    # Duplicate material override so each clone has independent shader uniforms
    if body and body.material_override:
        var dup = body.material_override.duplicate()
        if dup is ShaderMaterial:
            _mat = dup
            body.material_override = _mat
            _mat.set_shader_parameter("uMelt", 0.28)
            _reset_array_uniforms()

    if props == null:
        props = get_node_or_null("../Ecosystem") as SlimeProps
    if props:
        props.register_slime(self)

    _apply_transform()


func _exit_tree() -> void:
    if props and is_instance_valid(props):
        props.unregister_slime(self)


func _reset_array_uniforms() -> void:
    if _mat == null:
        return
    var zeros6 := PackedVector3Array()
    var small6 := PackedVector3Array()
    for i in 6:
        zeros6.append(Vector3.ZERO)
        small6.append(Vector3(0.01, 0.01, 0.01))
    var zeros2 := PackedVector3Array([Vector3.ZERO, Vector3.ZERO])
    var small2 := PackedVector3Array([Vector3(0.01, 0.01, 0.01), Vector3(0.01, 0.01, 0.01)])
    _mat.set_shader_parameter("uEatPos", zeros6)
    _mat.set_shader_parameter("uEatParam", zeros6)
    _mat.set_shader_parameter("uBoxHalf", small6)
    _mat.set_shader_parameter("uSolidPos", zeros2)
    _mat.set_shader_parameter("uSolidParam", zeros2)
    _mat.set_shader_parameter("uSolidHalf", small2)
    _mat.set_shader_parameter("uSolidDir", zeros2)
    _mat.set_shader_parameter("uDrapeAmt", 0.0)
    _mat.set_shader_parameter("uSplitAmt", 0.0)
    _mat.set_shader_parameter("uBudAmt", 0.0)
    _mat.set_shader_parameter("uBudScale", 0.0)


func _unhandled_input(event: InputEvent) -> void:
    if role != Role.PLAYER:
        return
    if event.is_action_pressed("slime_poke") and camera:
        if event is InputEventMouseButton or event is InputEventMouseMotion:
            _try_poke(event.position)
        elif event is InputEventScreenTouch:
            _try_poke(event.position)
        else:
            var vp := get_viewport()
            if vp:
                _try_poke(vp.get_visible_rect().size * 0.5)
    elif event.is_action_pressed("split_follow"):
        split_clone(Role.FOLLOWER)
    elif event.is_action_pressed("split_roam"):
        split_clone(Role.ROAMER)
    elif event.is_action_pressed("merge_clones"):
        merge_nearby_clones()


func _try_poke(screen_pos: Vector2) -> void:
    if camera == null:
        return
    var origin := camera.project_ray_origin(screen_pos)
    var dir := camera.project_ray_normal(screen_pos)
    var base_r := body_radius * slime_scale
    # Account for liquid flattening/spreading: raycast against true flattened ellipsoid shape
    var rx := base_r * lerpf(1.0, liquid_spread_ratio, liquid_amt)
    var ry := base_r * lerpf(1.0, liquid_height_ratio, liquid_amt)
    var rz := rx

    var inv_r := Vector3(1.0 / rx, 1.0 / ry, 1.0 / rz)
    var oc := (origin - global_position) * inv_r
    var d := dir * inv_r
    var a := d.dot(d)
    var b := oc.dot(d)
    var c := oc.dot(oc) - 1.0
    var disc := b * b - a * c
    if disc < 0.0:
        return
    var s := sqrt(disc)
    var t := (-b - s) / a
    if t < 0.0:
        t = (-b + s) / a
    if t < 0.0:
        return
    var hit := origin + dir * t
    var local_point := (hit - global_position).normalized() * body_radius
    var push := (hit - camera.global_position).normalized() * poke_strength
    push.y -= poke_downward
    trigger_poke(local_point, push)


func trigger_poke(local_point: Vector3, impulse: Vector3) -> void:
    _impulse_time = 0.001
    if _mat:
        _mat.set_shader_parameter("uImpulsePos", local_point)
        _mat.set_shader_parameter("uImpulseVector", impulse)
    poked.emit(local_point, impulse)


func trigger_poke2(local_point: Vector3, impulse: Vector3) -> void:
    _impulse2_time = 0.001
    if _mat:
        _mat.set_shader_parameter("uImpulse2Pos", local_point)
        _mat.set_shader_parameter("uImpulse2Vector", impulse)


func _process(delta: float) -> void:
    var dt := clampf(delta, 0.0, 0.1)
    _time += dt
    _impulse_time += dt
    _impulse2_time += dt

    # --- Merge animation (M key) ---
    if is_merging:
        merge_timer += dt
        if merge_target and is_instance_valid(merge_target) and not merge_target.is_merging:
            position = position.lerp(merge_target.position, 14.0 * dt)
            slime_scale = lerpf(slime_scale, 0.02, 12.0 * dt)
            _apply_transform()
            if merge_timer >= merge_duration or position.distance_squared_to(merge_target.position) < 0.2:
                if not _has_merged:
                    _has_merged = true
                    merge_target.target_scale = pow(pow(merge_target.target_scale, 3.0) + pow(merge_source_scale, 3.0), 1.0 / 3.0)
                    merge_target.trigger_poke(Vector3(0, 0.3, 0), Vector3(0, -0.65, 0))
                    merge_target.trigger_poke2(Vector3(0, -0.3, 0), Vector3(0, 0.5, 0))
                    # Safely transfer any ongoing digestion props to merge_target
                    if props:
                        for p in props.props:
                            if p.state == SlimeProps.STATE_EATING and p.eater == self:
                                p.eater = merge_target
                                # Fix 10: Avoid duplicate entries in merge target's digestion list
                                if not merge_target.inside_props.has(p):
                                    merge_target.inside_props.append(p)
                    inside_props.clear()
                    queue_free()
        else:
            # Target disappeared or became invalid: cancel merge safely instead of deleting self
            is_merging = false
            merge_target = null
            _has_merged = false
        return

    # ---- PARENT fission: budding separation ----
    if is_fissioning:
        fission_timer += dt
        var k: float = clampf(fission_timer / fission_duration, 0.0, 1.0)
        var e := sin(k * PI * 0.5)

        slime_scale = lerpf(fission_start_scale, fission_target_scale, e)
        _apply_transform()

        var stretch_amt := sin(k * PI) * 0.75
        if _mat:
```

```
            _mat.set_shader_parameter("uSplitAmt", stretch_amt)
            _mat.set_shader_parameter("uSplitDir", fission_fwd)

        if k >= 1.0:
            is_fissioning = false
            slime_scale = fission_target_scale
            if _mat:
                _mat.set_shader_parameter("uSplitAmt", 0.0)

    # ---- CLONE budding: emerging from parent ----
    if is_budding:
        bud_timer += dt
        var k: float = clampf(bud_timer / bud_duration, 0.0, 1.0)
        var e := sin(k * PI * 0.5)

        slime_scale = lerpf(bud_target_scale * 0.25, bud_target_scale, e)
        _apply_transform()

        if bud_parent and is_instance_valid(bud_parent):
            var sep_dist := (bud_parent.body_radius * bud_parent.slime_scale + body_radius * slime_scale) * 1.15
            position = bud_parent.position + bud_fwd * (sep_dist * e)
            position.y = bud_parent.position.y

        var stretch_amt := sin(k * PI) * 0.75
        if _mat:
            _mat.set_shader_parameter("uSplitAmt", stretch_amt)
            _mat.set_shader_parameter("uSplitDir", -bud_fwd)

        if k >= 1.0:
            is_budding = false
            bud_parent = null
            slime_scale = bud_target_scale
            if _mat:
                _mat.set_shader_parameter("uSplitAmt", 0.0)

    # --- Handle role-based input or AI driving ---
    var desired := Vector3.ZERO
    # Lock movement during clone snap animation; parent anchors/slows during viscous fission
    if not is_budding:
        match role:
            Role.PLAYER:
                desired = _process_player_input(dt)
                if is_fissioning:
                    desired *= 0.35
            Role.FOLLOWER:
                desired = _process_follower_ai(dt)
            Role.ROAMER:
                desired = _process_roamer_ai(dt)

    # --- Liquid transition ---
    var target_liquid := 1.0 if liquid_mode else 0.0
    liquid_amt += (target_liquid - liquid_amt) * (1.0 - exp(-12.0 * dt))
    if _mat:
        _mat.set_shader_parameter("uLiquid", liquid_amt)

    # --- Horizontal velocity update ---
    var v_resp := velocity_response * (0.6 if liquid_amt > 0.5 else 1.0)
    velocity = velocity.lerp(desired, 1.0 - exp(-v_resp * dt))
    position.x += velocity.x * dt
    position.z += velocity.z * dt

    # Clamp to arena boundary
    var flat := Vector2(position.x, position.z).length()
    if flat > arena_radius:
        position.x *= arena_radius / flat
        position.z *= arena_radius / flat

    # --- Scale smoothing (updated BEFORE vertical motion so ground height aligns with frame scale) ---
    slime_scale += (target_scale - slime_scale) * (1.0 - exp(-scale_response * dt))
    _apply_transform()

    # --- Vertical motion & climbing on props ---
    _process_vertical(dt)

    # --- Record breadcrumbs for followers ---
    if role == Role.PLAYER:
        _breadcrumb_timer += dt
        if _breadcrumb_timer >= 0.1:
            _breadcrumb_timer = 0.0
            # Fix 20: Record environment (ceiling height, gap width) instead of raw liquid state.
            # Followers use their own size to decide whether to flatten — not leader's Z state.
            var ceiling_h := _measure_ceiling_clearance()
            var gap_w := cur_gap_w
            breadcrumbs.append({"pos": position, "ceiling_h": ceiling_h, "gap_w": gap_w, "time": _time})
            if breadcrumbs.size() > 120:
                breadcrumbs.pop_front()

    # --- Crawl / lean springs ---
    var speed := velocity.length()
    speed01 = minf(speed / speed_for_full_crawl, 1.0)
    if speed > 0.05:
        _crawl_dir = Vector3(velocity.x, 0.0, velocity.z).normalized()
    # Heavy crawl struggle when straining against an edible object near slime size
    var crawl_strain := 1.0 - clampf(press_amt * 0.55, 0.0, 0.6)
    _crawl_phase += speed * dt * crawl_rate * crawl_strain

    var lean_target := Vector3(velocity.x * lean_gain * (1.0 - calm), 0.0, velocity.z * lean_gain * (1.0 - calm)) - _lean
    _lean_vel += lean_target * lean_stiffness * dt
    _lean_vel += _lean_vel * (-lean_damping * dt)
    _lean += _lean_vel * dt
    if _time - touch_time < 0.2:
        _lean_vel = Vector3.ZERO
        _lean *= exp(-14.0 * dt)

    # --- Eyes orientation & blink ---
    if speed > moving_threshold:
        var diff := atan2(_crawl_dir.x, _crawl_dir.z) - _facing_yaw
        diff = atan2(sin(diff), cos(diff))
        _facing_yaw += diff * (1.0 - exp(-facing_response * dt))
    var blink := blink_closed_amount if fmod(_time, blink_period) > blink_start else 1.0

    # --- Per-frame shader uniforms ---
    if _mat:
        _mat.set_shader_parameter("uImpulseTime", _impulse_time)
        _mat.set_shader_parameter("uImpulse2Time", _impulse2_time)
        _mat.set_shader_parameter("uMoveDir", _crawl_dir)
        _mat.set_shader_parameter("uMoveAmt", speed01 * (1.0 - 0.95 * calm))
        _mat.set_shader_parameter("uCrawl", _crawl_phase)
        _mat.set_shader_parameter("uLean", _lean)
        _mat.set_shader_parameter("uRippleIntensity", ripple_base + ripple_per_speed * speed01 + liquid_amt * 0.15 + press_amt * 0.28)
        _mat.set_shader_parameter("uFacing", _facing_yaw)
        _mat.set_shader_parameter("uBlink", blink)

    # --- Engulfing update ---
    # Fix 21: Followers flattening to fit under overhangs must NOT accidentally devour props
    if not (role == Role.FOLLOWER and _following_flatten):
        _process_engulf(dt)


func _process_player_input(dt: float) -> Vector3:
    # Block Z (liquid) during fission — combining deformations causes visual/physics conflicts
    var wants_liquid := Input.is_action_pressed("slime_liquid")
    if is_fissioning:
        liquid_mode = false
    else:
        liquid_mode = wants_liquid

    var in_z := Input.get_axis("move_back", "move_forward")
    var in_x := Input.get_axis("move_left", "move_right")

    var fwd := Vector3(0, 0, -1)
    if camera:
        fwd = -camera.global_transform.basis.z
        fwd.y = 0.0
        fwd = fwd.normalized()
    var right := fwd.cross(Vector3.UP)
    var desired := fwd * in_z + right * in_x

    if liquid_mode:
        cur_max = liquid_speed
    elif Input.is_action_pressed("run"):
        cur_max = run_speed
    else:
        cur_max = walk_speed

    if desired.length_squared() > 0.0:
        desired = desired.normalized() * cur_max
    return desired


func _process_follower_ai(dt: float) -> Vector3:
    # Fix 20–23: Environment breadcrumbs, own clearance decision, stuck detection, beeline cap
    if target_leader == null or not is_instance_valid(target_leader):
        target_leader = null
        if props and props.player_slime and is_instance_valid(props.player_slime) and props.player_slime != self:
            target_leader = props.player_slime
        else:
            _following_flatten = false
            return Vector3.ZERO

    # Determine individual slot offset so followers don't stack on top of each other
    var follower_index := 0
    if props:
        for s in props.registered_slimes:
            if s == self:
                break
            if is_instance_valid(s) and s.role == Role.FOLLOWER and s.target_leader == target_leader:
                follower_index += 1

    var slot_spacing := (slime_scale * body_radius * 2.0 + 0.8)
    var desired_dist := follow_distance * (target_leader.slime_scale + slime_scale) * 0.5 + float(follower_index) * slot_spacing

    var target_pt := target_leader.position
    # Fix 20: Use recorded environment to decide liquid — NOT the leader's liquid_mode directly
    var env_ceiling_h := INF
    var env_gap_w := INF

    var trail := target_leader.breadcrumbs
    var found_breadcrumb := false
    if trail.size() > 0:
        var accum := 0.0
        var prev := target_leader.position
        for i in range(trail.size() - 1, -1, -1):
            var b = trail[i]
            accum += prev.distance_to(b["pos"])
            prev = b["pos"]
            if accum >= desired_dist:
                target_pt = b["pos"]
                env_ceiling_h = b.get("ceiling_h", INF)
                env_gap_w = b.get("gap_w", INF)
                found_breadcrumb = true
                break

    if not found_breadcrumb:
        # Fallback: stay behind leader rather than collapsing onto leader's center
        var leader_back := -target_leader._crawl_dir
        if leader_back.length_squared() < 0.01:
            leader_back = Vector3(0, 0, 1)
        target_pt = target_leader.position + leader_back.normalized() * desired_dist

    var to_target := target_pt - position
    to_target.y = 0.0
    var dist := to_target.length()

    # Fix 22: Follower stuck detection
    _follower_stuck_timer += dt
    if _ai_last_pos.distance_squared_to(position) > 0.3:
        _follower_stuck_timer = 0.0
        _ai_last_pos = position
    if _follower_stuck_timer > 3.0 and dist > 1.0:
        # Pick an intermediate position offset to get unstuck
        var perp := to_target.cross(Vector3.UP).normalized()
        target_pt = position + perp * 3.0 + to_target.normalized() * 2.0
        _follower_stuck_timer = 0.0

    # Fix 20: Decide liquid from environment, not leader's state
    # Need clearance for our actual size, with lookahead (start flattening before reaching low spot)
    var my_full_height := height_per_scale * slime_scale * 2.0
    var needs_flat := env_ceiling_h < my_full_height * 1.1 or env_gap_w < body_radius * slime_scale * 1.8
    # Hysteresis: flatten when needed, only un-flatten when clearance is comfortable
    var has_room := env_ceiling_h > my_full_height * 1.4 and env_gap_w > body_radius * slime_scale * 2.5
    if needs_flat and not _following_flatten:
        _following_flatten = true
        liquid_mode = true
    elif _following_flatten and has_room:
        _following_flatten = false
        liquid_mode = false
    # Also flatten if our own squeeze detector says we're squeezing
    if squeeze_k < 0.65 and not _following_flatten:
        _following_flatten = true
        liquid_mode = true
    elif _following_flatten and squeeze_k > 0.85 and has_room:
        _following_flatten = false
        liquid_mode = false

    # Fix 23: Cap beeline speed — no tunneling through walls when catching up
    var base_speed := liquid_speed if liquid_mode else walk_speed
    var max_catch_up := run_speed * 0.9
    cur_max = max_catch_up if dist > 8.0 else base_speed

    # Smooth arrival easing to prevent stopping threshold jitter/overshoot
    var stop_dist := slime_scale * 0.35 + 0.2
    var slow_dist := stop_dist + 2.0
    if dist <= stop_dist:
        return Vector3.ZERO

    var speed_factor := clampf((dist - stop_dist) / (slow_dist - stop_dist), 0.0, 1.0)
    speed_factor = speed_factor * speed_factor * (3.0 - 2.0 * speed_factor)
    return to_target.normalized() * (cur_max * speed_factor)


func _process_roamer_ai(dt: float) -> Vector3:
    ## Fix 19: Pure wander — no food goal-seeking. Gentle heading drift with collision-driven liquid.
    _ai_roam_timer += dt
    _ai_stuck_timer += dt
    if _ai_recovery_timer > 0.0:
        _ai_recovery_timer = maxf(0.0, _ai_recovery_timer - dt)

    # Stuck detection
    if _ai_last_pos.distance_squared_to(position) > 0.2:
        _ai_stuck_timer = 0.0
        _ai_last_pos = position

    # Stuck recovery: pick an escape heading and go liquid briefly
    if _ai_stuck_timer > 2.5:
        if _ai_stuck_timer > 4.5:
            var ang := randf() * TAU
            var escape_dest := position + Vector3(cos(ang) * 12.0, 0, sin(ang) * 12.0)
            var flat := Vector2(escape_dest.x, escape_dest.z)
            if flat.length() > arena_radius * 0.85:
                flat = flat.normalized() * (arena_radius * 0.75)
                escape_dest.x = flat.x
                escape_dest.z = flat.y
            _ai_target_pos = escape_dest
            _ai_stuck_timer = 0.0
            _ai_recovery_timer = 2.5
        # Liquid during stuck, cleared below with hysteresis
        if not liquid_mode:
            liquid_mode = true

    # Liquid hysteresis: squeeze_k < 0.6 → go liquid; don't un-liquid until squeeze_k > 0.85
    # This prevents rapid flickering at the threshold.
    if squeeze_k < 0.6 and not liquid_mode:
        liquid_mode = true
```

```
    elif liquid_mode and squeeze_k > 0.85 and _ai_recovery_timer <= 0.0 and _ai_stuck_timer < 1.0:
        liquid_mode = false

    # Wander: pick a new gentle heading every 2–4 seconds
    if _ai_roam_timer > 0.0 and (position.distance_squared_to(_ai_target_pos) < 9.0 or _ai_target_pos == Vector3.ZERO):
        _ai_roam_timer = 0.0
        # Drift heading ±45° from current crawl direction rather than fully random long goals
        var base_ang := atan2(_crawl_dir.x, _crawl_dir.z) if _crawl_dir.length_squared() > 0.01 else randf() * TAU
        var drift := randf_range(-PI * 0.4, PI * 0.4)
        var new_ang := base_ang + drift
        var wander_dist := randf_range(6.0, 14.0)
        var wander_dest := position + Vector3(sin(new_ang) * wander_dist, 0.0, cos(new_ang) * wander_dist)
        var flat := Vector2(wander_dest.x, wander_dest.z)
        if flat.length() > arena_radius * 0.85:
            flat = flat.normalized() * (arena_radius * 0.75)
            wander_dest.x = flat.x
            wander_dest.z = flat.y
        _ai_target_pos = wander_dest
        # Randomize next heading change interval (2–4 s)
        _ai_roam_timer = -randf_range(2.0, 4.0)

    var wander_dir := (_ai_target_pos - position)
    wander_dir.y = 0.0
    var desired_heading := wander_dir.normalized() if wander_dir.length_squared() > 0.01 else _crawl_dir

    cur_max = liquid_speed if liquid_mode else walk_speed
    return desired_heading * cur_max


func _find_best_food_for_roamer():
    if props == null:
        return null
    var edible_lim := body_radius * slime_scale * 1.5
    var best_p = null
    var best_score := -99999.0

    for p in props.props:
        if p.state != SlimeProps.STATE_IDLE or p.get("is_static") or p.radius >= edible_lim:
            continue
        if p.mesh == null or not is_instance_valid(p.mesh):
            continue
        var dist := position.distance_to(p.mesh.position)
        if dist > 85.0:
            continue
        var score: float = (p.radius * 25.0) - dist * 1.2
        if score > best_score:
            best_score = score
            best_p = p
    return best_p


func _process_vertical(dt: float) -> void:
    # Node position stays firmly at support height; vertex shader flattens from the top down to the floor
    var S := slime_scale
    var full_target_y := support_y + (height_per_scale * S)

    # Fix 6: Before standing back up from liquid, check for overhead clearance.
    # If a prop is in the way, stay flat (liquid_clearance_ratio height) rather than getting ejected sideways.
    # Overhead clearance check:
    # Keep the base firmly anchored to support_y, and pass headroom compression to shader,
    # NEVER sink the node position.y into the floor!
    var target_y := full_target_y
    var overhead_compression := 1.0
    if liquid_amt > 0.05 and props != null:
        var full_top := support_y + (2.0 * height_per_scale * S)
        var flat_top := support_y + (2.0 * height_per_scale * S * liquid_height_ratio)
        var slime_r := body_radius * S
        var overhead_blocked := false
        for p in props.props:
            if p.state != SlimeProps.STATE_IDLE or p.mesh == null:
                continue
            var m := p.mesh
            var he := props._half_of(p)
            var prop_bottom := m.position.y - he.y
            if prop_bottom < flat_top or prop_bottom > full_top + 0.2:
                continue
            var dx := m.position.x - position.x
            var dz := m.position.z - position.z
            var dist_h := Vector2(dx, dz).length()
            if dist_h < slime_r + maxf(he.x, he.z) * 0.9:
                overhead_blocked = true
                var available_height := maxf(0.1, prop_bottom - support_y)
                overhead_compression = clampf(available_height / (2.0 * height_per_scale * S), liquid_height_ratio, 1.0)
                break
        if overhead_blocked and not liquid_mode and role == Role.PLAYER:
            liquid_mode = true

    set_shader_param("uOverheadCompress", overhead_compression)

    # Fix 25: When shrinking (target_scale < slime_scale), target_y drops and would trigger
    # gravity fall. Instead, lerp down smoothly to avoid a micro-bounce on every split/merge.
    var is_shrinking := target_scale < slime_scale - 0.01
    if position.y > target_y + 0.02 and not is_shrinking:
        is_on_ground = false
        vertical_vel -= 22.0 * dt
        position.y += vertical_vel * dt
        if position.y <= target_y:
            position.y = target_y
            vertical_vel = 0.0
            is_on_ground = true
    else:
        position.y = lerpf(position.y, target_y, 1.0 - exp(-16.0 * dt))
        vertical_vel = 0.0
        is_on_ground = true
    # In liquid mode the puddle still must stay at or above the floor — clamp it.
    var liquid_floor := floor_y + height_per_scale * S * liquid_height_ratio * 0.5
    if liquid_amt > 0.1 and position.y < liquid_floor:
        position.y = liquid_floor


func _process_engulf(dt: float) -> void:
    # Liquid mode (Z key) engulfing:
    # If covering an object with the liquid body from above or all around,
    # the slime drapes over and devours the object!
    # NO arbitrary radius cutoffs — eligibility is determined by physical coverage and volume budget!
    var target_p = null
    if liquid_mode and liquid_amt > 0.35 and props != null:
        var S := slime_scale
        var spread := lerpf(1.0, liquid_spread_ratio, liquid_amt)
        var puddle_radius := body_radius * S * spread
        # Actual liquid puddle bottom sits at support_y / floor_y:
        var puddle_bottom := support_y
        var my_volume := get_volume()

        # Check props: prioritize currently mounted ground_prop if any, else check nearby props
        var candidates: Array = []
        if ground_prop != null and ground_prop.state == SlimeProps.STATE_IDLE and not ground_prop.is_static:
            candidates.append(ground_prop)
        for p in props.props:
            if p.state == SlimeProps.STATE_IDLE and not p.is_static and p != ground_prop:
                candidates.append(p)

        var best_score := -INF
        for p in candidates:
            var m: MeshInstance3D = p.mesh
            if m == null or not is_instance_valid(m):
                continue
            var he: Vector3 = props._half_of(p)
            var prop_top: float = m.position.y + he.y
            var prop_bottom: float = m.position.y - he.y

            # Section 4 Volume budget: covering requires enough slime volume to form a shell
            # Shell area approx = top footprint + side walls
            var top_area := 4.0 * he.x * he.z if p.shape == SlimeProps.SHAPE_BOX else (PI * he.x * he.z)
            var side_area := (4.0 * he.x + 4.0 * he.z) * (2.0 * he.y)
            var min_shell_thickness := 0.08 * S
            var needed_shell_volume := (top_area + side_area) * min_shell_thickness
            if my_volume < needed_shell_volume * 0.8:
                continue # Slime doesn't have enough volume to cover this shape

            var flat_d: float = Vector2(position.x - m.position.x, position.z - m.position.z).length()

            # Coverage check against real footprint (corner / vertex reach):
            var prop_max_r: float = Vector2(he.x, he.z).length()
            var covers_horiz := (flat_d + prop_max_r <= puddle_radius * 1.05)

            # Vertical reach: slime must be atop or draping down over the prop
            var touches_vert := (puddle_bottom <= prop_top + 0.3 * S) and (position.y + 0.5 * S >= prop_bottom)

            if covers_horiz and touches_vert:
                var score: float = 1000.0 - flat_d
                if score > best_score:
                    best_score = score
                    target_p = p
                break # Found best valid candidate (prioritized ground_prop)

    if target_p != null:
        if engulf_target != target_p:
            engulf_target = target_p
            engulf_progress = 0.0

        engulf_progress = minf(1.0, engulf_progress + dt / maxf(devour_duration, 0.1))

        if _mat and target_p.mesh and is_instance_valid(target_p.mesh):
            var m: MeshInstance3D = target_p.mesh
            _mat.set_shader_parameter("uDrapePos", (m.position - position) / slime_scale)
            _mat.set_shader_parameter("uDrapeHalf", target_p.half * m.scale.x / slime_scale)
            _mat.set_shader_parameter("uDrapeShape", float(target_p.shape) + target_p.sides * 0.01)
            _mat.set_shader_parameter("uDrapeYaw", m.rotation.y)
            _mat.set_shader_parameter("uDrapeAmt", engulf_progress)

        if engulf_progress >= 1.0:
            _devour_prop(target_p)
            engulf_progress = 0.0
            engulf_target = null
            if _mat:
                _mat.set_shader_parameter("uDrapeAmt", 0.0)
        return

    # Target was lost or not covering: decay visual draping and progress
    if engulf_progress > 0.0:
        engulf_progress = maxf(0.0, engulf_progress - dt * 2.0)
        if _mat:
            _mat.set_shader_parameter("uDrapeAmt", engulf_progress)
        if engulf_progress <= 0.001:
            engulf_progress = 0.0
            engulf_target = null
            if _mat:
                _mat.set_shader_parameter("uDrapeAmt", 0.0)
    else:
        engulf_target = null


func _measure_ceiling_clearance() -> float:
    ## Returns the vertical clearance above this slime (distance to nearest overhead prop bottom).
    if props == null:
        return INF
    var S := slime_scale
    var slime_top := position.y + height_per_scale * S
    var slime_r := body_radius * S
    var closest := INF
    for p in props.props:
        if p.state != SlimeProps.STATE_IDLE or p.mesh == null:
            continue
        var m := p.mesh
        var he := props._half_of(p)
        var prop_bottom := m.position.y - he.y
        if prop_bottom <= slime_top:
            continue  # below or at our level
        var dx := m.position.x - position.x
        var dz := m.position.z - position.z
        if Vector2(dx, dz).length() < slime_r + maxf(he.x, he.z):
            var clearance := prop_bottom - slime_top
            if clearance < closest:
                closest = clearance
    return closest


func _devour_prop(p) -> void:
    p.state = SlimeProps.STATE_EATING
    p.eater = self
    p.base_scale = p.mesh.scale # store exact scale: can never grow bigger when digesting!
    p.t = 0.0
    p.start = p.mesh.position
    p.off = Vector3(randf() - 0.5, -0.1 + randf() * 0.3, randf() - 0.5)
    p.total_mass = pow(p.radius, 3.0) * 0.3
    p.absorbed_mass = 0.0
    inside_props.append(p)
    if props:
        props.eaten += 1
        props.eaten_changed.emit(props.eaten)


func split_clone(clone_role: Role) -> void:
    ## Split into two independent volume-conserved slimes (each = 0.7937x parent scale).
    # Block splits during liquid mode (Z), while transitioning back to solid, or while engulfing
    if target_scale < 0.45 or is_fissioning or is_budding or is_merging:
        return
    if liquid_mode or liquid_amt > 0.1 or engulf_target != null or Input.is_action_pressed("slime_liquid"):
        return

    # Fix 26: Use camera direction if slime is standing still, rather than stale _crawl_dir
    var fwd: Vector3
    if velocity.length() > 0.3:
        fwd = _crawl_dir.normalized()
    elif camera:
        fwd = -camera.global_transform.basis.z
        fwd.y = 0.0
        fwd = fwd.normalized()
    else:
        fwd = _crawl_dir.normalized()
    if fwd.length_squared() < 0.01:
        fwd = Vector3(0, 0, -1)

    var split_fwd := -fwd
    var new_target_scale := target_scale * pow(0.5, 1.0 / 3.0)

    # Spawn clone FIRST. If spawning fails, parent is completely untouched!
    var clone := _spawn_fission_clone(clone_role, split_fwd, new_target_scale)
    if clone == null:
        return

    # Only commit parent fission state after successful clone spawn
    fission_fwd = split_fwd
    fission_duration = 0.38
    fission_start_scale = slime_scale
    fission_target_scale = new_target_scale
    target_scale = new_target_scale
    is_fissioning = true
    fission_timer = 0.0


func _spawn_fission_clone(clone_role: Role, fwd_dir: Vector3, clone_target_scale: float) -> Slime:
    var prefab := load("res://scenes/slime.tscn") as PackedScene
    if prefab == null:
        return null
    var clone := prefab.instantiate() as Slime
    if clone == null:
        return null
```

```
    clone.role = clone_role
    clone.start_scale = clone_target_scale * 0.25
    clone.target_scale = clone_target_scale
    clone.slime_scale = clone.start_scale
    clone.props = props
    clone.floor_mesh = floor_mesh
    clone.floor_y = floor_y
    clone.support_y = support_y

    clone.position = position + fwd_dir * (0.2 * body_radius * clone_target_scale)
    clone.position.y = position.y
    clone._facing_yaw = _facing_yaw

    if clone_role == Role.FOLLOWER:
        clone.target_leader = self

    clone.is_budding = true
    clone.bud_parent = self
    clone.bud_timer = 0.0
    clone.bud_duration = 0.38
    clone.bud_target_scale = clone_target_scale
    clone.bud_fwd = fwd_dir

    var p_node := get_parent()
    if p_node:
        p_node.add_child(clone)
    else:
        return null

    return clone


func merge_nearby_clones() -> void:
    # Fix 8: Block merging during liquid/engulf states
    if props == null or is_merging or is_fissioning or is_budding:
        return
    if liquid_mode or liquid_amt > 0.1:
        return
    # Reduced merge range to prevent pulling clones through walls
    var merge_rad := (body_radius * slime_scale) * 2.5 + 1.0
    var to_merge: Array[Slime] = []

    for s in props.registered_slimes:
        if s == self or not is_instance_valid(s) or s.is_merging or s.is_fissioning or s.is_budding:
            continue
        # Fix 9: Skip clones in liquid state too (undefined interaction)
        if s.liquid_mode or s.liquid_amt > 0.1:
            continue
        var dist := position.distance_to(s.position)
        if dist > merge_rad + (s.body_radius * s.slime_scale):
            continue
        # Simple line-of-sight check: reject if any static wall AABB separates the two positions
        var blocked := false
        if props:
            var from2 := Vector2(position.x, position.z)
            var to2 := Vector2(s.position.x, s.position.z)
            for p in props.props:
                if not p.is_static or p.mesh == null:
                    continue
                var m := p.mesh
                var he := props._half_of(p)
                # AABB 2D segment intersection (simple rejection test)
                var bmin := Vector2(m.position.x - he.x, m.position.z - he.z)
                var bmax := Vector2(m.position.x + he.x, m.position.z + he.z)
                # Expand box slightly for safety
                bmin -= Vector2(0.1, 0.1)
                bmax += Vector2(0.1, 0.1)
                var seg := to2 - from2
                var t0 := 0.0
                var t1 := 1.0
                for axis in 2:
                    var o := from2[axis]
                    var d2 := seg[axis]
                    var lo := bmin[axis]
                    var hi := bmax[axis]
                    if absf(d2) < 1e-6:
                        if o < lo or o > hi:
                            t0 = 1.0  # parallel and outside
                            break
                    else:
                        var ta := (lo - o) / d2
                        var tb := (hi - o) / d2
                        if ta > tb:
                            var tmp := ta; ta = tb; tb = tmp
                        t0 = maxf(t0, ta)
                        t1 = minf(t1, tb)
                if t0 <= t1:
                    blocked = true
                    break
        if not blocked:
            to_merge.append(s)

    if to_merge.is_empty():
        return

    for s in to_merge:
        s.is_merging = true
        s.merge_target = self
        s.merge_timer = 0.0
        s.merge_duration = 0.42
        s.merge_source_scale = s.target_scale
        s.velocity = Vector3.ZERO

    trigger_poke(Vector3(0, 0.3, 0), Vector3(0, -0.6, 0))


func set_shader_param(param: StringName, value: Variant) -> void:
    if _mat:
        _mat.set_shader_parameter(param, value)


func _apply_transform() -> void:
    scale = Vector3.ONE * slime_scale
```

FILE: scripts/props.gd (1484 lines)

```
class_name SlimeProps
extends Node3D
## Port of the "3D Anime Slime" prop ecosystem.
##
## Spawns edible/solid props, manages multi-contact simultaneous collision resolution,
## fluid corner flow-around, directional squeeze & liquid mode hole clearance,
## climbing & engulfing on props, eating/digestion, force-driven splats, and multi-slime shadows.

signal eaten_changed(total: int)

const SHAPE_BOX := 0
const SHAPE_PRISM := 1
const SHAPE_CONE := 2

const STATE_IDLE := 0
const STATE_EATING := 1
const STATE_GONE := 2

const EAT_TIME := 0.8
const EAT_LINGER := 2.0

@export var slime: Slime
@export var floor_mesh: MeshInstance3D
@export var prop_shader: Shader
@export var prop_count := 36
@export var spawn_seed := 0
@export var floor_y := -1.2

@export_group("Squeeze & Liquid")
@export_range(0.06, 1.0, 0.01) var min_squeeze := 0.12
@export var squeeze_in_rate := 16.0
@export var squeeze_out_rate := 5.0

var props: Array[PropData] = []
var registered_slimes: Array[Slime] = []
var player_slime: Slime = null
var eaten := 0

var _rng := RandomNumberGenerator.new()
var _prop_root: Node3D
var _floor_mat: ShaderMaterial


class PropData extends RefCounted:
    var mesh: MeshInstance3D
    var material: ShaderMaterial
    var radius := 1.0
    var fp := 1.0
    var shape := 0
    var base_r := 1.0
    var sides := 0
    var phase := 0.0
    var half := Vector3.ONE
    var state := 0
    var t := 0.0
    var pd := 0.0
    var pv := 0.0
    var p_ro := 0.0
    var p_thr := 1.0
    var hold := 0.0
    var start := Vector3.ZERO
    var off := Vector3.ZERO
    var vel := Vector3.ZERO
    var touch := -1000.0
    var touch_slime: Slime = null
    var cnx := 0.0
    var cnz := 0.0
    var hit_cd := -1000.0
    var eater: Slime = null
    var last_presser: Slime = null
    var base_scale := Vector3.ONE
    var is_static := false
    var total_mass := 0.0
    var absorbed_mass := 0.0
    var real_volume := 0.0
    var engulf_front := 0.0 # Physical penetration depth allowed during push-eating (0.0 to 1.0)

    func calc_real_volume() -> float:
        var sx := mesh.scale.x if mesh else radius
        var sy := mesh.scale.y if mesh else radius
        var sz := mesh.scale.z if mesh else radius
        match shape:
            SHAPE_BOX:
                real_volume = (2.0 * half.x * sx) * (2.0 * half.y * sy) * (2.0 * half.z * sz)
            SHAPE_PRISM:
                var r := base_r * sx
                var h := 2.0 * half.y * sy
                if sides >= 3 and sides < 12:
                    var sector := TAU / float(sides)
                    var poly_area := 0.5 * float(sides) * r * r * sin(sector)
                    real_volume = poly_area * h
                else:
                    real_volume = PI * r * r * h
            SHAPE_CONE:
                var r := base_r * sx
                var h := 2.0 * half.y * sy
                real_volume = (1.0 / 3.0) * PI * r * r * h
            _:
                real_volume = (4.0 / 3.0) * PI * pow(radius, 3.0)
        total_mass = real_volume
        return real_volume



func _ready() -> void:
    # Run after Slimes (priority 0) update their positions, so collisions resolve before render
    process_priority = 1

    if prop_shader == null:
        prop_shader = load("res://shaders/prop.gdshader")
    if spawn_seed == 0:
        _rng.randomize()
    else:
        _rng.seed = spawn_seed

    if floor_mesh == null:
        floor_mesh = get_node_or_null("../Floor") as MeshInstance3D
    if floor_mesh:
        _floor_mat = floor_mesh.material_override as ShaderMaterial

    _prop_root = Node3D.new()
    _prop_root.name = "Generated"
    add_child(_prop_root)

    # Spawn test structures FIRST so random props avoid blocking the test gate and bridge!
    _spawn_test_structures()
    _spawn_props()

    if slime != null:
        register_slime(slime)


func register_slime(s: Slime) -> void:
    if not registered_slimes.has(s):
        registered_slimes.append(s)
        if s.role == Slime.Role.PLAYER:
            player_slime = s


func unregister_slime(s: Slime) -> void:
    registered_slimes.erase(s)
    if player_slime == s:
        player_slime = null
    for p in props:
        if p.last_presser == s:
            p.last_presser = null
            p.pd = 0.0
            p.pv = 0.0
            p.hold = 0.0
        if p.touch_slime == s:
            p.touch_slime = null


# ---------------------------------------------------------
# Spawning
# ---------------------------------------------------------

func _spawn_props() -> void:
    var makers := [
        {"shape": SHAPE_BOX, "size": Vector3(1.4, 1.4, 1.4)},
        {"shape": SHAPE_BOX, "size": Vector3(1.4, 1.4, 1.4)},
        {"shape": SHAPE_BOX, "size": Vector3(2.0, 0.9, 1.2)},
        {"shape": SHAPE_BOX, "size": Vector3(1.0, 2.2, 1.0)},
        {"shape": SHAPE_PRISM, "r": 0.8, "h": 1.6, "sides": 6},
        {"shape": SHAPE_PRISM, "r": 0.8, "h": 1.2, "sides": 20},
        {"shape": SHAPE_PRISM, "r": 0.9, "h": 1.4, "sides": 3},
        {"shape": SHAPE_CONE, "r": 1.0, "h": 1.8, "sides": 4, "yaw": PI / 4.0},
        {"shape": SHAPE_CONE, "r": 1.0, "h": 1.8, "sides": 16},
    ]

    for i in prop_count:
        var maker: Dictionary = makers[_rng.randi_range(0, makers.size() - 1)]
        var shape: int = maker["shape"]
        var mesh: Mesh
        var half: Vector3
        var base_r := 1.0
        var sides := 0
        var phase := 0.0
        var yaw_offset: float = maker.get("yaw", 0.0)

        match shape:
            SHAPE_BOX:
                var size: Vector3 = maker["size"]
                var box := BoxMesh.new()
                box.size = size
                mesh = box
                half = size * 0.5
            SHAPE_PRISM:
                base_r = maker["r"]
                var h: float = maker["h"]
                sides = maker["sides"]
                var prism := CylinderMesh.new()
                prism.top_radius = base_r
                prism.bottom_radius = base_r
                prism.height = h
                prism.radial_segments = sides
                prism.rings = 1
                mesh = prism
                half = Vector3(base_r, h * 0.5, base_r)
            SHAPE_CONE:
                base_r = maker["r"]
                var h: float = maker["h"]
                sides = maker["sides"]
                var cone := CylinderMesh.new()
                cone.top_radius = 0.0
                cone.bottom_radius = base_r
                cone.height = h
                cone.radial_segments = sides
                cone.rings = 1
                mesh = cone
                half = Vector3(base_r, h * 0.5, base_r)

        var radius := 0.4 + pow(_rng.randf(), 1.6) * 2.8
        var fp := radius * 1.25
        var position := Vector3.ZERO
        var placed := false

        for _tries in 80:
            var ang := _rng.randf() * TAU
            var dist := (6.0 + _rng.randf() * 12.0) if i < 12 else (16.0 + _rng.randf() * 60.0)
            var x := cos(ang) * dist
```

```
            var z := sin(ang) * dist
            if Vector2(x, z).length() < fp + 4.0:
                continue
            var clear := true
            for q in props:
                if Vector2(q.mesh.position.x - x, q.mesh.position.z - z).length() < q.fp + fp + 0.6:
                    clear = false
                    break
            if not clear:
                continue
            position = Vector3(x, floor_y + half.y * radius, z)
            placed = true
            break

        if not placed:
            continue

        var p := PropData.new()
        p.radius = radius
        p.fp = fp
        p.shape = shape
        p.base_r = base_r
        p.sides = sides
        p.phase = phase
        p.half = half
        p.state = STATE_IDLE

        var mat := ShaderMaterial.new()
        mat.shader = prop_shader
        mat.set_shader_parameter("uColor", _hsl(_rng.randf(), 0.65, 0.62))
        mat.set_shader_parameter("uDissolve", 0.0)
        p.material = mat

        var mi := MeshInstance3D.new()
        mi.name = "Prop%02d" % i
        mi.mesh = mesh
        mi.material_override = mat
        mi.scale = Vector3.ONE * radius
        mi.position = position
        mi.rotation.y = _rng.randf() * TAU + yaw_offset
        _prop_root.add_child(mi)
        p.mesh = mi
        p.base_scale = mi.scale
        p.calc_real_volume()
        props.append(p)


func _spawn_test_structures() -> void:
    var pillar_mat := ShaderMaterial.new()
    pillar_mat.shader = prop_shader
    pillar_mat.set_shader_parameter("uColor", Color(0.32, 0.72, 0.88))
    pillar_mat.set_shader_parameter("uDissolve", 0.0)

    # 1. Narrow Slit Gate: Two vertical pillars with a 0.5m gap at (8, 0, 8)
    var gate_pos := Vector3(8.0, 0.0, 8.0)
    for side in [-1.0, 1.0]:
        var p_box := BoxMesh.new()
        p_box.size = Vector3(1.2, 3.2, 1.2)
        var p := PropData.new()
        p.radius = 2.4
        p.fp = 2.4
        p.shape = SHAPE_BOX
        p.half = p_box.size * 0.5
        p.material = pillar_mat.duplicate() as ShaderMaterial
        var mi := MeshInstance3D.new()
        mi.name = "GatePillar_%s" % ("L" if side < 0 else "R")
        mi.mesh = p_box
        mi.material_override = p.material
        mi.position = gate_pos + Vector3(side * 0.85, floor_y + 1.6, 0.0)
        _prop_root.add_child(mi)
        p.mesh = mi
        p.is_static = true
        p.base_scale = mi.scale
        p.calc_real_volume()
        props.append(p)

    # 2. Low Tunnel / Overhang Bridge: Clearance of 0.75m at (-10, 0, 8)
    var bridge_pos := Vector3(-10.0, 0.0, 8.0)
    var roof_box := BoxMesh.new()
    roof_box.size = Vector3(3.2, 0.6, 2.4)
    var roof_prop := PropData.new()
    roof_prop.is_static = true
    roof_prop.radius = 2.8
    roof_prop.fp = 2.8
    roof_prop.shape = SHAPE_BOX
    roof_prop.half = roof_box.size * 0.5
    roof_prop.material = pillar_mat.duplicate() as ShaderMaterial
    var roof_mi := MeshInstance3D.new()
    roof_mi.name = "BridgeRoof"
    roof_mi.mesh = roof_box
    roof_mi.material_override = roof_prop.material
    roof_mi.position = bridge_pos + Vector3(0.0, floor_y + 0.75 + 0.3, 0.0)
    _prop_root.add_child(roof_mi)
    roof_prop.mesh = roof_mi
    roof_prop.calc_real_volume()
    props.append(roof_prop)

    for leg_side in [-1.0, 1.0]:
        var leg_box := BoxMesh.new()
        leg_box.size = Vector3(0.5, 0.75, 2.4)
        var leg_p := PropData.new()
        leg_p.is_static = true
        leg_p.radius = 1.8
        leg_p.fp = 1.8
        leg_p.shape = SHAPE_BOX
        leg_p.half = leg_box.size * 0.5
        leg_p.material = pillar_mat.duplicate() as ShaderMaterial
        var leg_mi := MeshInstance3D.new()
        leg_mi.name = "BridgeLeg_%s" % ("L" if leg_side < 0 else "R")
        leg_mi.mesh = leg_box
        leg_mi.material_override = leg_p.material
        leg_mi.position = bridge_pos + Vector3(leg_side * 1.35, floor_y + 0.375, 0.0)
        _prop_root.add_child(leg_mi)
        leg_p.mesh = leg_mi
        leg_p.calc_real_volume()
        props.append(leg_p)


func _hsl(h: float, s: float, l: float) -> Color:
    if s <= 0.0:
        return Color(l, l, l)
    var q := l * (1.0 + s) if l < 0.5 else l + s - l * s
    var p := 2.0 * l - q
    return Color(_hue(p, q, h + 1.0 / 3.0), _hue(p, q, h), _hue(p, q, h - 1.0 / 3.0))


func _hue(p: float, q: float, t: float) -> float:
    if t < 0.0:
        t += 1.0
    if t > 1.0:
        t -= 1.0
    if t < 1.0 / 6.0:
        return p + (q - p) * 6.0 * t
    if t < 0.5:
        return q
    if t < 2.0 / 3.0:
        return p + (q - p) * (2.0 / 3.0 - t) * 6.0
    return p


# ---------------------------------------------------------------
# Per-frame physics & ecosystem update
# ---------------------------------------------------------------

func _process(delta: float) -> void:
    var dt := minf(delta, 0.05)

    for s in registered_slimes:
        if is_instance_valid(s):
            _step_slime(s, dt)

    # Global press decay for any props no longer actively pressed by a valid slime
    for p in props:
        if p.state == STATE_IDLE and p.pd > 0.0:
            if p.last_presser == null or not is_instance_valid(p.last_presser):
                p.pd = maxf(0.0, p.pd * exp(-15.0 * dt))
                p.pv = 0.0
                if p.pd <= 0.001:
                    p.pd = 0.0
                    p.last_presser = null
                    p.hold = 0.0

    _update_digesting_props(dt)
    _update_floor_shadows()


func _step_slime(s: Slime, dt: float) -> void:
    if s.is_merging:
        return
    # 1. Detect if standing atop a solid prop or climbing
    _update_ground_support(s)

    # 2. Update squeeze & liquid mode gap passage
    _update_slime_squeeze(s, dt)

    # 3. Process edible props pressed/eaten by this slime
    _process_edible_props(s, dt)

    # 4. Multi-contact simultaneous solver for solid obstacles
    _solve_solid_collisions(s, dt)

    # 5. Splat & spread spring dynamics
    _update_slime_splat(s, dt)

    # 6. Inside swallowed prop physics for this slime
    _update_inside_props(s, dt)

    # 7. Feed eating and solid dent uniforms to this slime's shader
    _update_eat_uniforms(s)
    _update_solid_uniforms(s)


func _update_ground_support(s: Slime) -> void:
    var sp := s.position
    var S := s.slime_scale
    var highest_top := floor_y
    var found_prop = null
    var slime_bottom := sp.y - (s.height_per_scale * S)
    var slime_top := slime_bottom + (2.0 * s.height_per_scale * S) * lerpf(1.0, s.liquid_height_ratio, s.liquid_amt)

    for p in props:
        if p.state != STATE_IDLE:
            continue
        var m := p.mesh
        var he := _half_of(p)
        var dx := sp.x - m.position.x
        var dz := sp.z - m.position.z
        var top_y := m.position.y + he.y
        var bottom_y := m.position.y - he.y

        # If prop is elevated above the floor (roof/ceiling/bridge) and slime is under it, ignore
        if bottom_y > floor_y + 0.15 and slime_top <= bottom_y + 0.2:
            continue

        # Check if slime is horizontally within the prop's top footprint
        var in_footprint := false
        if p.shape == SHAPE_BOX:
            var c := cos(m.rotation.y)
            var sn := sin(m.rotation.y)
            var lx := absf(dx * c + dz * sn)
            var lz := absf(-dx * sn + dz * c)
            in_footprint = (lx <= he.x * 1.05 and lz <= he.z * 1.05)
        else:
            var d := Vector2(dx, dz).length()
            var sup := _poly_support(p, dx / maxf(d, 1e-4), dz / maxf(d, 1e-4))
            in_footprint = (d <= sup * 1.05)

        if in_footprint:
            # Slime stepping/climbing: step height scales with slime size (0.55 * S) or liquid climbing (0.75 * S)
            var climb_reach := (0.75 * S) if s.liquid_amt > 0.3 else (0.55 * S)
            if slime_bottom >= top_y - climb_reach:
                if top_y > highest_top:
                    highest_top = top_y
                    found_prop = p

    s.support_y = highest_top
    s.ground_prop = found_prop


func _prop_closest_surface_point(p: PropData, target: Vector2) -> Vector2:
    var m := p.mesh
    var he := _half_of(p)
    var dx := target.x - m.position.x
    var dz := target.y - m.position.z
    if p.shape == SHAPE_BOX:
        var c := cos(m.rotation.y)
        var sn := sin(m.rotation.y)
        var lx := dx * c - dz * sn
        var lz := dx * sn + dz * c
        var cx := clampf(lx, -he.x, he.x)
        var cz := clampf(lz, -he.z, he.z)
        if absf(cx) < he.x and absf(cz) < he.z:
            var push_x := he.x - absf(cx)
            var push_z := he.z - absf(cz)
            if push_x < push_z:
                cx = he.x if cx >= 0.0 else -he.x
            else:
                cz = he.z if cz >= 0.0 else -he.z
        var wx := cx * c + cz * sn
        var wz := -cx * sn + cz * c
        return Vector2(m.position.x + wx, m.position.z + wz)
    else:
        var d := maxf(Vector2(dx, dz).length(), 1e-4)
        var ux := dx / d
        var uz := dz / d
        var sup := _poly_support(p, ux, uz) * (0.6 if p.shape == SHAPE_CONE else 1.0)
        return Vector2(m.position.x + ux * sup, m.position.z + uz * sup)


func _update_slime_squeeze(s: Slime, dt: float) -> void:
    var S := s.slime_scale
    var sp := s.position

    # Fix 17: Don't allow liquid to flatten to paper-thin (0.05). Use 0.12 minimum.
    var effective_min_squeeze := min_squeeze
    if s.liquid_amt > 0.01:
        effective_min_squeeze = lerpf(min_squeeze, 0.12, s.liquid_amt)

    var r_max := 0.0
    for k in 6:
        r_max = maxf(r_max, _surf_r(-1.2 + 2.2 * float(k) / 5.0))
    r_max *= S
    var reach := r_max + 0.06 * S

    var slime_bottom := sp.y - s.height_per_scale * S
    var slime_top := slime_bottom + (2.0 * s.height_per_scale * S) * lerpf(1.0, s.liquid_height_ratio, s.liquid_amt)

    var nearby_props: Array[PropData] = []
    for p in props:
        if p.state != STATE_IDLE or p == s.ground_prop or p.mesh == null:
            continue
        # Fix 18: Exclude elevated static overhangs (bridge roof/legs) from gap pairing —
        # they are ceilings, not side walls, and create false near-zero gaps.
        var m := p.mesh
        var he := _half_of(p)
        var prop_bottom := m.position.y - he.y
        if p.is_static and prop_bottom > floor_y + 0.5:
            continue
        if maxf(m.position.y - he.y, floor_y) > minf(m.position.y + he.y, slime_top):
            continue
        var d_sq := Vector2(m.position.x - sp.x, m.position.z - sp.z).length_squared()
        var p_rad := maxf(he.x, he.z)
        if d_sq > (p_rad + reach * 3.0) * (p_rad + reach * 3.0):
            continue
        nearby_props.append(p)

    var spx := Vector2(sp.x, sp.z)
    var best_w := INF
    var best_dir := Vector2.ZERO
    var best_center := Vector2.ZERO
    var best_dist := INF
    var best_flow_dir := Vector2.ZERO

    for i in nearby_props.size():
        var p1 := nearby_props[i]
        var c1 := Vector2(p1.mesh.position.x, p1.mesh.position.z)
        for j in range(i + 1, nearby_props.size()):
            var p2 := nearby_props[j]
            var c2 := Vector2(p2.mesh.position.x, p2.mesh.position.z)
            var pt1 := _prop_closest_surface_point(p1, c2)
            var pt2 := _prop_closest_surface_point(p2, pt1)
            pt1 = _prop_closest_surface_point(p1, pt2)

            var gap_vec := pt2 - pt1
            var gap_w := gap_vec.length()
            if gap_w < 1e-3 or gap_w >= best_w:
```

```
                continue
            var gap_dir := gap_vec / gap_w
            var gap_center := (pt1 + pt2) * 0.5
            var flow_dir := Vector2(-gap_dir.y, gap_dir.x)

            var to_gap := spx - gap_center
            var dist_along_flow := absf(to_gap.dot(flow_dir))
            var dist_lateral := absf(to_gap.dot(gap_dir))

            if dist_along_flow < reach * 2.5 and dist_lateral < (gap_w * 0.5 + reach * 1.5):
                best_w = gap_w
                best_dir = gap_dir
                best_center = gap_center
                best_dist = to_gap.length()
                best_flow_dir = flow_dir

    s.cur_gap_w = best_w
    s.cur_gap_center = best_center
    s.cur_gap_dir = best_dir

    var target := 1.0
    if best_w < INF:
        var fit := clampf((best_w * 0.5 - 0.02 * S) / r_max, effective_min_squeeze, 1.0)
        if fit < 1.0:
            var to_gap_vec := best_center - spx
            var dist_flow := to_gap_vec.x * best_flow_dir.x + to_gap_vec.y * best_flow_dir.y
            var dist_lat := absf(to_gap_vec.x * best_dir.x + to_gap_vec.y * best_dir.y)

            var corridor_half_len := 0.6 * S
            var entrance_flow := dist_flow - corridor_half_len # distance from slime center to entrance
            var exit_flow := dist_flow + corridor_half_len     # distance from slime center to exit

            var max_lat := best_w * 0.5 + reach * 0.8
            if dist_lat < max_lat:
                if entrance_flow > 0.0 and entrance_flow < reach * 1.5:
                    # Slime front reaches entrance: squeeze smoothly from 1.0 down to fit
                    var t_enter := clampf((reach * 1.5 - entrance_flow) / (reach * 1.2), 0.0, 1.0)
                    t_enter = t_enter * t_enter * (3.0 - 2.0 * t_enter)
                    var lat_factor := clampf(1.0 - dist_lat / max_lat, 0.0, 1.0)
                    target = lerpf(1.0, fit, t_enter * lat_factor)
                elif entrance_flow <= 0.0 and exit_flow > -reach * 1.2:
                    # Inside corridor: maintain fit squeeze until back exits
                    target = fit
                elif exit_flow <= -reach * 1.2:
                    # Fully exited corridor: snap back to round
                    target = 1.0

    # Fix 16: Once slime has fully passed through, release squeeze quickly using squeeze_in_rate
    var rate: float
    if target < s.squeeze_k:
        rate = squeeze_in_rate  # squeezing in (tightening)
    else:
        # Releasing — use squeeze_in_rate if the body is clearly past the gap to snap back
        var past_gap := best_w == INF or absf((spx - best_center).dot(best_flow_dir)) > reach * 1.5
        rate = squeeze_in_rate if past_gap else squeeze_out_rate
    s.squeeze_k = clampf(s.squeeze_k + (target - s.squeeze_k) * (1.0 - exp(-rate * dt)), effective_min_squeeze, 1.0)

    if target < 0.98 and best_dir.length_squared() > 0.001:
        var d3 := Vector3(best_dir.x, 0.0, best_dir.y).normalized()
        if d3.dot(s.sq_dir) < 0.0:
            d3 = -d3
        s.sq_dir = s.sq_dir.lerp(d3, 1.0 - exp(-16.0 * dt)).normalized()
        if s.sq_dir.length_squared() < 0.01:
            s.sq_dir = Vector3(1, 0, 0)

        # Smooth funneling force: gently guide slime toward the gap centerline as it moves
        var lateral_offset := (spx - best_center).dot(best_dir)
        var to_gap := Vector3(best_center.x - sp.x, 0.0, best_center.y - sp.z)
        var moving_toward := s.velocity.dot(to_gap) > -0.5 or to_gap.length() < reach * 2.0
        if moving_toward and (spx - best_center).length() < reach * 2.5:
            var dist_to_gap := (spx - best_center).length()
            var dist_factor := clampf(1.0 - dist_to_gap / (reach * 2.2), 0.0, 1.0)
            var align_factor := clampf(1.0 - absf(lateral_offset) / (reach * 1.8), 0.0, 1.0) * maxf(0.35, dist_factor)
            # Guide firmly along gap centerline
            var funnel_push := -Vector3(best_dir.x, 0.0, best_dir.y) * lateral_offset * align_factor * 12.0 * dt
            s.position += funnel_push
            s.velocity -= Vector3(best_dir.x, 0.0, best_dir.y) * lateral_offset * align_factor * 4.0 * dt

    var squeeze_val := 1.0 - s.squeeze_k
    s.set_shader_param("uSqueeze", squeeze_val)
    s.set_shader_param("uSqueezeDir", s.sq_dir)


func is_edible_by_push(s: Slime, p: PropData) -> bool:
    if p.state != STATE_IDLE or p.is_static or p.mesh == null:
        return false
    # Authoritative volume check: Slime can push-eat anything whose real volume is <= Slime's real volume
    return s.get_volume() >= p.real_volume * 0.98


func _process_edible_props(s: Slime, dt: float) -> void:
    var S := s.slime_scale
    var sp := s.position
    var s_vol := s.get_volume()

    for p in props:
        if not is_edible_by_push(s, p):
            # If this prop was being pressed by this slime but is no longer push-edible, decay its engulf_front
            if p.last_presser == s:
                p.engulf_front = maxf(0.0, p.engulf_front - dt * 2.0)
                continue
        var m := p.mesh
        var dx := m.position.x - sp.x
        var dz := m.position.z - sp.z
        var d := maxf(Vector2(dx, dz).length(), 1e-4)

        var he0 := _half_of(p)
        var ca := cos(m.rotation.y)
        var sa := sin(m.rotation.y)
        var ro: float
        if p.shape == SHAPE_BOX:
            ro = absf((dx * ca - dz * sa) / d) * he0.x + absf((dx * sa + dz * ca) / d) * he0.z
        else:
            ro = _poly_support(p, -dx / d, -dz / d) * (0.6 if p.shape == SHAPE_CONE else 1.0)

        var sr_p := _surf_r((m.position.y - sp.y) / S) * S

        var target := maxf(0.0, sr_p + ro - d)
        if target <= 0.001:
            # If this slime was pressing this prop, smoothly decay its press spring & engulf front back out
            if p.last_presser == s:
                p.engulf_front = maxf(0.0, p.engulf_front - dt * 2.5)
                var acc := 140.0 * (0.0 - p.pd) - 8.0 * p.pv
                p.pv += acc * dt
                p.pd += p.pv * dt
                if p.pd <= 0.001 and p.engulf_front <= 0.001:
                    p.pd = 0.0
                    p.pv = 0.0
                    p.engulf_front = 0.0
                    p.last_presser = null
                    p.hold = 0.0
            continue

        # If another valid slime is already pressing this prop and is closer, don't steal it
        if p.last_presser != null and p.last_presser != s and is_instance_valid(p.last_presser):
            var dist_other := p.mesh.position.distance_to(p.last_presser.position)
            var dist_me := p.mesh.position.distance_to(sp)
            if dist_other < dist_me:
                continue

        p.last_presser = s
        # Compute exact relative volume ratio (prop volume / slime volume)
        var vol_ratio := clampf(p.real_volume / maxf(1e-4, s_vol), 0.0, 1.0)

        var thr := ro * (0.85 + 0.45 * vol_ratio) + 0.1 * S
        var press01 := minf(target / thr, 1.0)
        var acc := 140.0 * (target - p.pd) - 8.0 * p.pv
        p.pv += acc * dt
        p.pd += p.pv * dt
        if p.pd < 0.0:
            p.pd = 0.0
            if p.pv < 0.0:
                p.pv = 0.0
        p.p_ro = ro
        p.p_thr = thr

        # Section 1: Physical Engulf-Front Resistance
        # Instead of a velocity cap that running bypasses, constrain slime's physical position
        # to a moving engulf front that advances only with sustained pushing!
        var nx0 := dx / d
        var nz0 := dz / d
        var inward_speed := s.velocity.x * nx0 + s.velocity.z * nz0

        # Resistance factor: 0.0 for tiny items, ramping up steeply for objects near slime size
        var size_resistance := clampf((vol_ratio - 0.20) / 0.80, 0.0, 1.0)

        # 1. Viscous Speed Drag:
        # Pushing into objects near slime size heavily resists forward movement down to a crawl
        if inward_speed > 0.0:
            var allowed_speed := lerpf(s.cur_max, s.cur_max * 0.15, pow(size_resistance, 1.4))
            if inward_speed > allowed_speed:
                var excess := inward_speed - allowed_speed
                s.velocity.x -= nx0 * excess
                s.velocity.z -= nz0 * excess
                inward_speed = allowed_speed

        # 2. Advance Engulf Front:
        # Tiny props (< 20%): fast gulp (rate ~ 3.5 - 4.5)
        # Medium props (50%): steady strain (rate ~ 1.0)
        # Props near slime size (> 80%): heavy resistance, steep slowdown (rate ~ 0.06 - 0.2)
        if inward_speed > 0.1:
            var size_factor := pow(maxf(0.0, 1.0 - vol_ratio), 1.8)
            var advance_rate := 3.8 * size_factor + 0.06 * (1.0 - size_resistance * 0.85)
            p.engulf_front = minf(1.0, p.engulf_front + advance_rate * dt)
        else:
            # Elastic push back: if player lets go or stops pushing,
            # an object near slime size pushes back forcefully!
            var pushback_rate := 1.2 + 3.0 * size_resistance
            p.engulf_front = maxf(0.0, p.engulf_front - dt * pushback_rate)
            var recoil := 2.5 * size_resistance * (1.0 - p.engulf_front)
            s.velocity.x -= nx0 * recoil * dt
            s.velocity.z -= nz0 * recoil * dt

        # 3. Physical Positional Constraint:
        # Slime body cannot penetrate deeper than allowed by current engulf_front!
        var max_allowed_pen := (sr_p + ro) * p.engulf_front
        var current_pen := target
        if current_pen > max_allowed_pen:
            var over_pen := current_pen - max_allowed_pen
            # Constrain position so slime cannot push through before engulf front opens
            s.position.x -= nx0 * over_pen * 0.95
            s.position.z -= nz0 * over_pen * 0.95
            # Zero out inward penetration velocity against the engulf front wall
            var vin := s.velocity.x * nx0 + s.velocity.z * nz0
            if vin > 0.0:
                s.velocity.x -= nx0 * vin
                s.velocity.z -= nz0 * vin

        # Absorption completes when engulf front has fully enclosed the prop (>= 0.95)
        if p.engulf_front >= 0.95:
            p.state = STATE_EATING
            p.eater = s
            p.last_presser = null
            p.base_scale = m.scale
            eaten += 1
            p.t = 0.0
            p.start = m.position
            p.off = Vector3(_rng.randf() - 0.5, -0.1 + _rng.randf() * 0.3, _rng.randf() - 0.5)
            p.total_mass = p.real_volume
            p.absorbed_mass = 0.0
            eaten_changed.emit(eaten)


# ---------------------------------------------------------------------------
# Multi-contact simultaneous solver & fluid corner flow-around
# ---------------------------------------------------------------------------

class ContactInfo extends RefCounted:
    var prop: PropData
    var normal: Vector3
    var pen: float
    var c_r: float
    var sharp: float


func _solve_solid_collisions(s: Slime, dt: float) -> void:
    var S := s.slime_scale
    # Gather contacts from all solid props:
    # A prop is SOLID if it is NOT edible by push and is not our current ground support!
    var contacts: Array[ContactInfo] = []
    for p in props:
        if p.state != STATE_IDLE or p == s.ground_prop:
            continue
        # If it's edible by push, it's handled by engulf-front resistance, NOT solid collision solver!
        if is_edible_by_push(s, p):
            continue
        var c := _detect_prop_contact(s, p)
        if c != null and c.pen > 0.0:
            contacts.append(c)

    if contacts.is_empty():
        return

    # Sort deepest penetration first
    contacts.sort_custom(func(a: ContactInfo, b: ContactInfo) -> bool: return a.pen > b.pen)

    # Simultaneous iterative resolution (4 passes)
    for _pass in 4:
        if contacts.size() >= 2 and (contacts[0].normal.dot(contacts[1].normal) < -0.2 or contacts[0].prop != contacts[1].prop):
            var c0 := contacts[0]
            var c1 := contacts[1]
            var pen0 := _calc_prop_penetration(s, c0.prop, c0.c_r)
            var pen1 := _calc_prop_penetration(s, c1.prop, c1.c_r)
            # Symmetrically center the slime directly between opposing walls
            var net_push := (c0.normal * pen0 + c1.normal * pen1) * 0.5
            # When between two different props (corridor passage), only push laterally — don't push backward along flow!
            if c0.prop != c1.prop:
                var lat_axis := s.sq_dir if s.sq_dir.length_squared() > 0.01 else (c0.normal - c1.normal).normalized()
                net_push = lat_axis * net_push.dot(lat_axis)
                # Also adapt squeeze immediately upon physical double-wall contact
                if s.cur_gap_w < INF:
                    var fit_k := clampf((s.cur_gap_w * 0.5) / maxf(0.1, c0.c_r), 0.08, 1.0)
                    s.squeeze_k = minf(s.squeeze_k, lerpf(s.squeeze_k, fit_k, 0.4))
```

```
            if net_push.length_squared() > 1e-6:
                s.position.x += net_push.x
                s.position.z += net_push.z
            # Resolve any remaining non-opposing contacts (e.g. end wall)
            for idx in range(2, contacts.size()):
                var c := contacts[idx]
                var pen := _calc_prop_penetration(s, c.prop, c.c_r)
                if pen > 0.001:
                    s.position.x += c.normal.x * pen * 0.8
                    s.position.z += c.normal.z * pen * 0.8
        else:
            for c in contacts:
                var pen := _calc_prop_penetration(s, c.prop, c.c_r)
                if pen > 0.001:
                    var push := c.normal * pen * 0.75
                    if s.cur_gap_w < INF and s.sq_dir.length_squared() > 0.01:
                        # Never let single-wall lateral push overshoot past the gap centerline
                        var lat_axis := Vector3(s.sq_dir.x, 0.0, s.sq_dir.y)
                        var lat_push := push.dot(lat_axis)
                        var cur_offset := (Vector2(s.position.x, s.position.z) - s.cur_gap_center).dot(Vector2(lat_axis.x, lat_axis.z))
                        if (cur_offset > 0.0 and lat_push < 0.0) or (cur_offset < 0.0 and lat_push > 0.0):
                            var max_push := absf(cur_offset)
                            if absf(lat_push) > max_push:
                                push -= lat_axis * (lat_push - signf(lat_push) * max_push)
                    s.position += push

    # Primary contact for splat & stick
    var c0 := contacts[0]
    s.impact_dir = Vector3(-c0.normal.x, 0, -c0.normal.z)
    c0.prop.touch = s.time
    c0.prop.touch_slime = s
    c0.prop.cnx = c0.normal.x
    c0.prop.cnz = c0.normal.z
    s.sharp_amt += (c0.sharp - s.sharp_amt) * (1.0 - exp(-15.0 * dt))
    s.contact_r = c0.c_r / S
    s.touch_time = s.time

    # Multi-contact channel / narrow passage flow
    if contacts.size() >= 2:
        var c1 := contacts[1]
        if c0.prop != c1.prop:
            # Between two separate obstacles: treat as a corridor passage regardless of normal angle!
            var corridor_dir: Vector3
            if s.sq_dir.length_squared() > 0.01:
                corridor_dir = Vector3(-s.sq_dir.z, 0.0, s.sq_dir.x).normalized()
            else:
                var n_across := (c0.normal - c1.normal).normalized()
                corridor_dir = Vector3(-n_across.z, 0.0, n_across.x).normalized()

            var input_dir := s.velocity
            if input_dir.length_squared() < 0.01:
                input_dir = s._crawl_dir  # Slime has no .mesh; use _crawl_dir which is always valid
            if corridor_dir.dot(input_dir) < 0.0:
                corridor_dir = -corridor_dir

            var v_corridor := maxf(0.0, s.velocity.dot(corridor_dir))
            # Ensure forward velocity is maintained when pushing through
            if v_corridor < 1.0:
                v_corridor = maxf(v_corridor, s.velocity.length())
            s.velocity = corridor_dir * v_corridor
            return
        else:
            # Two contacts on the same prop: concave corner / crease on a single obstacle
            var dot_n := c0.normal.dot(c1.normal)
            if dot_n < 0.85:
                var sum_n := c0.normal + c1.normal
                if sum_n.length_squared() > 0.01:
                    var crease := sum_n.cross(Vector3.UP).normalized()
                    if crease.dot(s.velocity) < 0.0:
                        crease = -crease
                    var v_along_crease := maxf(0.0, s.velocity.dot(crease))
                    s.velocity = crease * v_along_crease
                    return

    # Single contact handling: fluid sliding with low friction
    var nx := c0.normal.x
    var nz := c0.normal.z
    var v := s.velocity
    var vdot := v.x * nx + v.z * nz

    if vdot < 0.0:
        var vin := -vdot
        # Project velocity onto surface tangent: cancel normal penetration velocity
        v.x -= nx * vdot
        v.z -= nz * vdot

        var vt_len := Vector2(v.x, v.z).length()
        if vt_len > 1e-4:
            var tangent := Vector3(v.x / vt_len, 0.0, v.z / vt_len)
            if s.squeeze_k < 0.95 or c0.sharp > 0.15:
                # Near openings or around corners: zero friction, slight fluid flow deflection around corner
                # Cap flow_boost to 0.15x to prevent runaway speed at triangle/cone vertices
                var flow_boost := vin * minf(0.15, (0.35 if s.squeeze_k < 0.95 else (0.25 * c0.sharp)))
                s.velocity = tangent * (vt_len + flow_boost)
            else:
                # Flat wall impact with viscous drag clinging
                var fr := (1.0 - exp(-6.0 * dt))
                s.velocity = tangent * maxf(0.0, vt_len * (1.0 - fr))
        else:
            # Head-on impact (vt_len <= 1e-4)
            # Deflect along tangent towards the gap opening so slime slides smoothly into the opening
            var tangent := Vector3(-nz, 0.0, nx)
            if s.cur_gap_w < INF:
                var to_gap := Vector3(s.cur_gap_center.x - s.position.x, 0.0, s.cur_gap_center.y - s.position.z)
                if tangent.dot(to_gap) < 0.0:
                    tangent = -tangent
                s.velocity = tangent * maxf(vin * 0.8, 2.0)
            else:
                # Use crawl_dir as stable tiebreak — to_prop is always perpendicular and flips each frame
                if tangent.dot(s._crawl_dir) < 0.0:
                    tangent = -tangent
                s.velocity = tangent * maxf(vin * 0.2, 0.5)

        # Only trigger impact poke/splat if hitting a flat wall in open space, NOT around corners or when squeezing
        if vin > 0.6 and s.squeeze_k > 0.85 and c0.sharp < 0.2 and s.time - c0.prop.hit_cd > 0.3:
            c0.prop.hit_cd = s.time
            var sev := minf(1.0, pow(vin / 8.5, 0.9))
            s.splat_v += sev * 28.0 * (1.0 + 0.5 * c0.sharp)
            s.trigger_poke(
                Vector3(-nx, -0.15, -nz).normalized() * 1.6,
                Vector3(nx * 1.6 * sev, 0.18 * sev, nz * 1.6 * sev)
            )


func _detect_prop_contact(s: Slime, p: PropData) -> ContactInfo:
    var S := s.slime_scale
    var sp := s.position
    var m := p.mesh
    var dx := m.position.x - sp.x
    var dz := m.position.z - sp.z
    var d := maxf(Vector2(dx, dz).length(), 1e-4)
    var he := _half_of(p)

    if d - Vector2(he.x, he.z).length() > 2.8 * S:
        return null

    var base_y := m.position.y - he.y
    var top_y := m.position.y + he.y
    var y_lo := maxf(base_y, floor_y)
    var slime_bottom := sp.y - s.height_per_scale * S
    var slime_effective_top := slime_bottom + (2.0 * s.height_per_scale * S) * lerpf(1.0, s.liquid_height_ratio, s.liquid_amt)
    var y_hi := minf(top_y, slime_effective_top)
    if y_hi < y_lo:
        return null

    var pen := 0.0
    var nx := 0.0
    var nz := 0.0
    var c_r := 0.0
    var sharp := 0.0
    var n := 5

    # Directional ellipse axes
    var sq_axis := s.sq_dir
    var flow_axis := Vector3(-sq_axis.z, 0, sq_axis.x)

    if p.shape == SHAPE_BOX:
        var r := 0.0
        for k in n:
            var y := y_lo + (y_hi - y_lo) * float(k) / float(n - 1)
            r = maxf(r, _surf_r((y - sp.y) / S) * S + 0.06 * S)

        var c := cos(m.rotation.y)
        var sn := sin(m.rotation.y)
        var wx := -dx
        var wz := -dz
        var lx := wx * c - wz * sn
        var lz := wx * sn + wz * c
        var cx := clampf(lx, -he.x, he.x)
        var cz := clampf(lz, -he.z, he.z)
        var ex := lx - cx
        var ez := lz - cz
        var dd := Vector2(ex, ez).length()

        # Corner sharpness
        var ox := absf(lx) - he.x
        var oz := absf(lz) - he.z
        var t := clampf(minf(ox, oz) / (0.22 * r), 0.0, 1.0)
        sharp = t * t * (3.0 - 2.0 * t)

        # Elliptical radius along contact direction
        var world_ex := ex * c + ez * sn
        var world_ez := -ex * sn + ez * c
        var dir_len := maxf(Vector2(world_ex, world_ez).length(), 1e-4)
        var unx := world_ex / dir_len
        var unz := world_ez / dir_len

        var r_gap := maxf(r * s.squeeze_k, 0.05 * S)
        # Only stretch along flow when aligned and with in the gap corridor,
        # never when approaching or hitting outside front faces!
        var gap_corridor_factor := 1.0
        if s.cur_gap_w < INF and sq_axis.length_squared() > 0.01:
            var to_gap := Vector2(sp.x - s.cur_gap_center.x, sp.z - s.cur_gap_center.y)
            var dist_flow := absf(to_gap.x * flow_axis.x + to_gap.y * flow_axis.z)
            gap_corridor_factor = clampf(1.0 - maxf(0.0, dist_flow - he.z) / (0.8 * S), 0.0, 1.0)
        var r_flow := maxf(r * (1.0 + 0.5 * (1.0 - s.squeeze_k) * gap_corridor_factor), 0.05 * S)
        var effective_r := r
        if sq_axis.length_squared() > 0.01:
            var p_gap := absf(unx * sq_axis.x + unz * sq_axis.z)
            var p_flow := absf(unx * flow_axis.x + unz * flow_axis.z)
            var denom := sqrt((p_gap * r_flow) * (p_gap * r_flow) + (p_flow * r_gap) * (p_flow * r_gap))
            if denom > 1e-4:
                effective_r = (r_gap * r_flow) / denom
            effective_r = clampf(effective_r, minf(r_gap, r_flow), maxf(r_gap, r_flow))
        c_r = effective_r

        if dd < effective_r:
            if dd > 1e-4:
                ex /= dd
                ez /= dd
                pen = effective_r - dd
            else:
                # Deep inside box: safely push opposing velocity
                var px := he.x - absf(lx)
                var pz := he.z - absf(lz)
                if px < pz:
                    ex = 1.0 if lx >= 0.0 else -1.0
                    ez = 0.0
                    pen = px + r_gap
                else:
                    ex = 0.0
                    ez = 1.0 if lz >= 0.0 else -1.0
                    pen = pz + r_flow
            nx = ex * c + ez * sn
            nz = -ex * sn + ez * c

            # When squeezing, side walls and corners must not push backward along the corridor flow
            if s.squeeze_k < 0.90:
                var flow_proj := Vector3(nx, 0.0, nz).dot(flow_axis)
                if flow_proj < 0.0:
                    var lateral_comp := Vector3(nx, 0.0, nz) - flow_axis * flow_proj
                    if lateral_comp.length_squared() > 1e-4:
                        var lat_norm := lateral_comp.normalized()
                        nx = lat_norm.x
                        nz = lat_norm.z
    else:
        var ux := -dx / d
        var uz := -dz / d
        var sup := _poly_support(p, ux, uz)

        if p.sides >= 3 and p.sides < 12:
            var rc := p.base_r * m.scale.x
            var ap := rc * cos(PI / p.sides)
            var t := clampf((sup - ap) / (rc - ap), 0.0, 1.0)
            sharp = t * t * (3.0 - 2.0 * t)

        var h := 2.0 * he.y
        for k in n:
            var y := y_lo + (y_hi - y_lo) * float(k) / float(n - 1)
            var rs := _surf_r((y - sp.y) / S) * S + 0.06 * S
            var r_gap := maxf(rs * s.squeeze_k, 0.05 * S)
            var r_flow := maxf(rs * (1.0 + 0.5 * (1.0 - s.squeeze_k)), 0.05 * S)
            var eff_rs := rs
            if sq_axis.length_squared() > 0.01:
                var p_gap := absf(ux * sq_axis.x + uz * sq_axis.z)
                var p_flow := absf(ux * flow_axis.x + uz * flow_axis.z)
                var denom := sqrt((p_gap * r_flow) * (p_gap * r_flow) + (p_flow * r_gap) * (p_flow * r_gap))
                if denom > 1e-4:
                    eff_rs = (r_gap * r_flow) / denom
            eff_rs = clampf(eff_rs, minf(r_gap, r_flow), maxf(r_gap, r_flow))
            var reach := sup * (maxf(0.06, 1.0 - (y - base_y) / h) if p.shape == SHAPE_CONE else 1.0)
            if eff_rs + reach - d > pen:
                pen = eff_rs + reach - d
                c_r = eff_rs
        if p.sides >= 3 and p.sides < 12:
            var ca := cos(m.rotation.y)
            var sa := sin(m.rotation.y)
            var lx := ux * ca - uz * sa
            var lz := ux * sa + uz * ca
            var sector := TAU / p.sides
            var phi := atan2(lx, lz) - p.phase
            var d_sec := fmod(phi, sector)
            if d_sec < 0.0:
                d_sec += sector
            var face_local_ang := atan2(lx, lz) - d_sec + sector * 0.5
            var fn_lx := sin(face_local_ang)
            var fn_lz := cos(face_local_ang)
            var fn_wx := fn_lx * ca + fn_lz * sa
            var fn_wz := -fn_lx * sa + fn_lz * ca
            # Blend between flat face normal and vertex radial normal based on sharpness
            var blend_to_radial := smoothstep(0.4, 0.9, sharp)
            nx = lerpf(fn_wx, ux, blend_to_radial)
            nz = lerpf(fn_wz, uz, blend_to_radial)
            var nlen := maxf(Vector2(nx, nz).length(), 1e-4)
            nx /= nlen
            nz /= nlen
        else:
            nx = ux
            nz = uz

        if s.squeeze_k < 0.90:
            var flow_proj := Vector3(nx, 0.0, nz).dot(flow_axis)
```

```
            if flow_proj < 0.0:
                var lateral_comp := Vector3(nx, 0.0, nz) - flow_axis * flow_proj
                if lateral_comp.length_squared() > 1e-4:
                    var lat_norm := lateral_comp.normalized()
                    nx = lat_norm.x
                    nz = lat_norm.z

    if pen <= 0.0:
        return null

    var ci := ContactInfo.new()
    ci.prop = p
    ci.normal = Vector3(nx, 0, nz).normalized()
    ci.pen = pen
    ci.c_r = c_r
    ci.sharp = sharp
    return ci


func _calc_prop_penetration(s: Slime, p: PropData, c_r: float) -> float:
    var sp := s.position
    var S := s.slime_scale
    var m := p.mesh
    var dx := m.position.x - sp.x
    var dz := m.position.z - sp.z
    var he := _half_of(p)

    if p.shape == SHAPE_BOX:
        var c := cos(m.rotation.y)
        var sn := sin(m.rotation.y)
        var lx := -dx * c + dz * sn
        var lz := -dx * sn - dz * c
        var cx := clampf(lx, -he.x, he.x)
        var cz := clampf(lz, -he.z, he.z)
        var dd := Vector2(lx - cx, lz - cz).length()
        return maxf(0.0, c_r - dd)
    else:
        var d := maxf(Vector2(dx, dz).length(), 1e-4)
        var ux := -dx / d
        var uz := -dz / d
        var base_y := m.position.y - he.y
        var top_y := m.position.y + he.y
        var h := 2.0 * he.y
        var slime_bottom := sp.y - s.height_per_scale * S
        var slime_effective_top := slime_bottom + (2.0 * s.height_per_scale * S) * lerpf(1.0, s.liquid_height_ratio, s.liquid_amt)
        var y_lo := maxf(base_y, floor_y)
        var y_hi := minf(top_y, slime_effective_top)
        # For cones: use the same per-height tapered reach as detection, to avoid stutter
        if p.shape == SHAPE_CONE and h > 1e-4:
            var best_pen := 0.0
            var n := 5
            for k in n:
                var y := y_lo + (y_hi - y_lo) * float(k) / float(n - 1)
                var sup_full := _poly_support(p, ux, uz)
                var reach := sup_full * maxf(0.06, 1.0 - (y - base_y) / h)
                var rs := _surf_r((y - sp.y) / S) * S + 0.06 * S
                var pen_at_y := rs + reach - d
                if pen_at_y > best_pen:
                    best_pen = pen_at_y
            return maxf(0.0, best_pen)
        else:
            var sup := _poly_support(p, ux, uz)
            return maxf(0.0, c_r + sup - d)

func _update_slime_splat(s: Slime, dt: float) -> void:
    var since := s.time - s.touch_time
    # Sustained push into a wall keeps the slime flattened
    var is_pushing := s.velocity.length_squared() > 0.1
    var tgt := 0.20 if since < 0.15 else (0.12 if (since < 0.4 and is_pushing) else 0.0)

    var ks := 120.0 if s.splat_v > 0.0 else 40.0
    s.splat_v += (ks * (tgt - s.splat_x) - 2.0 * sqrt(ks) * s.splat_v) * dt
    s.splat_x = clampf(s.splat_x + s.splat_v * dt, 0.0, 1.1)
    if s.splat_x == 0.0 and s.splat_v < 0.0:
        s.splat_v = 0.0

    var kp := 70.0 if s.splat_x > s.spread_x else 22.0
    s.spread_v += (kp * (s.splat_x - s.spread_x) - 2.0 * sqrt(kp) * s.spread_v) * dt
    s.spread_x = maxf(0.0, s.spread_x + s.spread_v * dt)
    if s.spread_x == 0.0 and s.spread_v < 0.0:
        s.spread_v = 0.0

    s.set_shader_param("uSplat", s.splat_x * (1.0 - 0.7 * s.sharp_amt))
    s.set_shader_param("uSpread", s.spread_x * (1.0 - 0.5 * s.sharp_amt))
    s.set_shader_param("uWall", s.contact_r)
    s.set_shader_param("uSplatDir", s.impact_dir)


func _update_digesting_props(dt: float) -> void:
    for p in props:
        if p.state == STATE_EATING:
            p.t += dt
            # Smoothly decay press spring so the contact dent fades during absorption
            p.pd = maxf(0.0, p.pd * exp(-10.0 * dt))
            p.pv = maxf(0.0, p.pv * exp(-10.0 * dt))
            var k := minf(p.t / EAT_TIME, 1.0)
            var e := k * k * (3.0 - 2.0 * k)
            var dis := maxf(0.0, 1.0 - maxf(0.0, p.t - EAT_TIME) / EAT_LINGER)
            var shrink := maxf(0.001, (1.0 - 0.35 * e) * (0.65 + 0.35 * dis))
            p.mesh.scale = p.base_scale * shrink
            if p.material != null:
                p.material.set_shader_parameter("uDissolve", 1.0 - dis)
            p.mesh.rotation.y += dt * 0.4


            # Smoothly transfer digested mass into slime size progression
            var s: Slime = p.eater if p.eater != null and is_instance_valid(p.eater) else player_slime
            if s != null and is_instance_valid(s):
                var total_digest_time: float = EAT_TIME + EAT_LINGER
                var progress := clampf(p.t / total_digest_time, 0.0, 1.0)
                var e_mass := progress * progress * (3.0 - 2.0 * progress)
                var target_absorbed: float = p.total_mass * e_mass
                var delta_m: float = target_absorbed - p.absorbed_mass
                if delta_m > 0.0:
                    p.absorbed_mass = target_absorbed
                    # Authoritative volume transfer: add prop volume directly into slime target scale
                    # Volume = (4/3)*PI*(body_radius*target_scale)^3
                    # target_scale_new = (target_scale^3 + delta_m / ((4/3)*PI*body_radius^3))^(1/3)
                    var vol_factor := (4.0 / 3.0) * PI * pow(s.body_radius, 3.0)
                    s.target_scale = pow(pow(s.target_scale, 3.0) + (delta_m / vol_factor), 1.0 / 3.0)

                var S: float = s.slime_scale
                if p.t < EAT_TIME:
                    var target_in_belly: Vector3 = s.position + p.off * (S * 0.70)
                    p.mesh.position = p.start.lerp(target_in_belly, e)
                    p.vel = Vector3.ZERO
                else:
                    if not s.inside_props.has(p):
                        s.inside_props.append(p)

            if dis <= 0.0:
                if p.eater != null and is_instance_valid(p.eater):
                    p.eater.inside_props.erase(p)
                elif player_slime != null:
                    player_slime.inside_props.erase(p)
                p.mesh.queue_free()
                p.state = STATE_GONE


func _update_inside_props(s: Slime, dt: float) -> void:
    # Inside props physics for this slime
    var S := s.slime_scale
    var sp := s.position
    for item in s.inside_props:
        var p: PropData = item as PropData
        if p == null or p.mesh == null:
            continue
        p.vel.y -= 7.0 * dt
        p.vel *= exp(-2.0 * dt)
        p.mesh.position += p.vel * dt

        var m: MeshInstance3D = p.mesh
        var he: Vector3 = _half_of(p)
        var rr: float = maxf(maxf(he.x, he.y), he.z) * 0.8
        var rh: float = maxf(0.15, 1.6 * S - rr)
        var rv: float = maxf(0.15, 1.15 * S - rr)
        var l: Vector3 = m.position - sp
        var qx: float = l.x / rh
        var qy: float = l.y / rv
        var qz: float = l.z / rh
        var ql: float = sqrt(qx * qx + qy * qy + qz * qz)
        if ql > 1.0:
            m.position = sp + l / ql
            var n: Vector3 = Vector3(qx / rh, qy / rv, qz / rh).normalized()
            var vn: float = p.vel.dot(n)
            if vn > 0.0:
                p.vel -= n * vn * 1.4

        if m.position.y < floor_y + he.y:
            m.position.y = floor_y + he.y
            if p.vel.y < 0.0:
                p.vel.y *= -0.2


func _update_eat_uniforms(s: Slime) -> void:
    var S := s.slime_scale
    var sp := s.position
    var eat_pos := PackedVector3Array()
    var eat_param := PackedVector3Array()
    var box_half := PackedVector3Array()
    for i in 6:
        eat_pos.append(Vector3.ZERO)
        eat_param.append(Vector3.ZERO)
        box_half.append(Vector3(0.01, 0.01, 0.01))

    var pressing: Array[PropData] = []
    for p in props:
        # Fix: Only IDLE props occupy pressing slots (0-2). Once EATING, props occupy eating slots (3-5) exclusively,
        # preventing double-dent deformation in the shader during the first 0.8s.
        if p.state == STATE_IDLE and p.pd > 0.004 and p.p_ro > 0.0 and p.last_presser == s:
            var dist := Vector2(p.mesh.position.x - sp.x, p.mesh.position.z - sp.z).length()
            if dist <= 2.2 * S + p.p_ro:
                pressing.append(p)
    pressing.sort_custom(func(a: PropData, b: PropData) -> bool: return a.pd > b.pd)
    if pressing.size() > 3:
        pressing.resize(3)

    var pa := 0.0
    var press_dir := Vector3(0, 0, 1)
    for i in pressing.size():
        var p := pressing[i]
        var m := p.mesh.position
        var dx := m.x - sp.x
        var dz := m.z - sp.z
        var d := maxf(Vector2(dx, dz).length(), 1e-4)
        var yl := (m.y - sp.y) / S
        var dv := (_surf_r(yl) * S + p.p_ro - p.pd) / S
        eat_pos[i] = Vector3(dx / d * dv, yl, dz / d * dv)
        eat_param[i] = Vector3(float(p.shape) + p.sides * 0.01, 1.0, p.mesh.rotation.y + p.phase)
        box_half[i] = _box_half(p, S)
        var a := minf(1.0, p.pd / p.p_thr)
        var vr := clampf(p.real_volume / maxf(1e-4, s.get_volume()), 0.0, 1.0)
        var strain_boost := 1.0 + 0.45 * clampf((vr - 0.25) / 0.75, 0.0, 1.0)
        a = minf(1.0, a * strain_boost)
        if a > pa:
            pa = a
            press_dir = Vector3(dx / d, 0, dz / d)

    s.press_amt += (pa - s.press_amt) * 0.25
    s.set_shader_param("uPressAmt", s.press_amt)
    s.set_shader_param("uPressDir", press_dir)

    var near := 0.0
    for p in props:
        # Only count props that are actively being pressed by this slime (p_ro is stale otherwise)
        if p.state != STATE_IDLE or p.p_ro <= 0.0 or p.last_presser != s:
            continue
        var gap := Vector2(p.mesh.position.x - sp.x, p.mesh.position.z - sp.z).length() - (1.9 * S + p.p_ro)
        near = maxf(near, minf(1.0, maxf(0.0, 1.0 - gap / (1.5 * S + p.p_ro))))
    s.calm += (maxf(near, s.press_amt) - s.calm) * 0.2

    var slot := 3
    for p in props:
        if p.state != STATE_EATING or p.eater != s or slot > 5:
            continue
        var m := p.mesh.position
        eat_pos[slot] = Vector3((m.x - sp.x) / S, (m.y - sp.y) / S, (m.z - sp.z) / S)
        var eat_fade := clampf(p.t / 0.4, 0.0, 1.0)
        eat_param[slot] = Vector3(float(p.shape) + p.sides * 0.01, eat_fade, p.mesh.rotation.y + p.phase)
        box_half[slot] = _box_half(p, S)
        slot += 1

    s.set_shader_param("uEatPos", eat_pos)
    s.set_shader_param("uEatParam", eat_param)
    s.set_shader_param("uBoxHalf", box_half)


func _update_solid_uniforms(s: Slime) -> void:
    var S := s.slime_scale
    var sp := s.position
    var solid_pos := PackedVector3Array()
    var solid_param := PackedVector3Array()
    var solid_half := PackedVector3Array()
    var solid_dir := PackedVector3Array()
    for i in 2:
        solid_pos.append(Vector3.ZERO)
        solid_param.append(Vector3.ZERO)
        solid_half.append(Vector3(0.01, 0.01, 0.01))
        solid_dir.append(Vector3.ZERO)

    var active: Array[PropData] = []
    for p in props:
        if p.state == STATE_IDLE and p.touch_slime == s and s.time - p.touch < 1.2:
            var he := _half_of(p)
            var dist := Vector2(p.mesh.position.x - sp.x, p.mesh.position.z - sp.z).length()
            if dist <= 2.4 * S + Vector2(he.x, he.z).length():
                active.append(p)
    active.sort_custom(func(a: PropData, b: PropData) -> bool: return a.touch > b.touch)
    if active.size() > 2:
        active.resize(2)

    var depth := minf(1.2, maxf(s.splat_x, s.spread_x * 0.5) * (0.5 + 0.7 * s.sharp_amt))
    for i in active.size():
        var p := active[i]
        var m := p.mesh.position
        solid_pos[i] = Vector3((m.x - sp.x) / S + p.cnx * depth, (m.y - sp.y) / S, (m.z - sp.z) / S + p.cnz * depth)
        solid_dir[i] = Vector3(-p.cnx, 0, -p.cnz)
        var fade := 1.0 - minf(1.0, maxf(0.0, (s.time - p.touch - 0.8) / 0.4))
        solid_param[i] = Vector3(float(p.shape) + p.sides * 0.01, fade, p.mesh.rotation.y + p.phase)
        solid_half[i] = _box_half(p, S)

    s.set_shader_param("uSolidPos", solid_pos)
    s.set_shader_param("uSolidParam", solid_param)
    s.set_shader_param("uSolidHalf", solid_half)
    s.set_shader_param("uSolidDir", solid_dir)


func _update_floor_shadows() -> void:
    if _floor_mat == null:
        return
    var b_pos := PackedVector3Array()
    var b_scale := PackedFloat32Array()

    for s in registered_slimes:
        if b_pos.size() >= 8:
            break
        if not is_instance_valid(s):
            continue
        var s_scale := s.slime_scale * lerpf(1.0, 1.85, s.liquid_amt)
        b_pos.append(s.global_position)
        b_scale.append(s_scale)



    var count := b_pos.size()
    while b_pos.size() < 8:
        b_pos.append(Vector3.ZERO)
        b_scale.append(0.0)

    _floor_mat.set_shader_parameter("uBlobCount", count)
    _floor_mat.set_shader_parameter("uBlobPos", b_pos)
    _floor_mat.set_shader_parameter("uBlobScale", b_sca
```

```
le)


# ------------------------------------------------------
# Shape helpers
# ------------------------------------------------------

func _surf_r(yl: float) -> float:
    var melt := 0.28
    var k := 0.86 - melt * 0.22
    var g_floor := -1.6 * k
    var lo := g_floor
    var hi := 1.6 * k
    for _i in 18:
        var mid := (lo + hi) * 0.5
        var b := mid - g_floor
        var f := mid
        if b < 0.9:
            var t := clampf(b / 0.9, 0.0, 1.0)
            f = g_floor + 0.9 * (2.0 * t * t - t * t * t)
        if f < yl:
            lo = mid
        else:
            hi = mid

    var py := clampf(lo / k, -1.6, 1.6)
    var h := (py / 1.6 + 1.0) * 0.5
    var sm := smoothstep(0.0, 0.22, h)
    var spread := 1.0 + (pow(1.0 - h, 1.35) * sm * 0.34 + sin(h * PI) * 0.08) * (0.8 + melt * 1.1)
    return maxf(0.3, sqrt(maxf(0.0, 2.56 - py * py)) * spread)


func _half_of(p: PropData) -> Vector3:
    return p.half * p.mesh.scale.x


func _poly_support(p: PropData, ux: float, uz: float) -> float:
    var r := p.base_r * p.mesh.scale.x
    if p.sides < 3 or p.sides >= 12:
        return r
    var ca := cos(p.mesh.rotation.y)
    var sa := sin(p.mesh.rotation.y)
    var lx := ux * ca - uz * sa
    var lz := ux * sa + uz * ca
    var sector := TAU / p.sides
    var d := fmod(atan2(lx, lz) - p.phase, sector)
    if d < 0.0:
        d += sector
    var from_face := absf(d - sector * 0.5)
    return r * cos(sector * 0.5) / cos(from_face)


func _box_half(p: PropData, S: float) -> Vector3:
    var he := _half_of(p)
    if p.shape == SHAPE_BOX:
        return he / S
    var r := p.base_r * p.mesh.scale.x
    return Vector3(r, he.y, r) / S


func total() -> int:
    return props.size()
```

FILE: scripts/orbit_camera.gd (91 lines)

```
class_name OrbitCamera
extends Node3D
## Drag-to-orbit, wheel-zoom camera that follows the slime. Mirrors the HTML's cameraState.

@export var camera: Camera3D
@export var follow: Slime

@export_group("Orbit")
@export var theta := 0.45
@export var phi := 1.18
@export var radius := 5.4
@export var drag_sensitivity := 0.007
@export var min_phi := 0.35
@export_range(0.0, 3.14159, 0.01) var max_phi := 1.5080

@export_group("Zoom")
@export var zoom_step := 0.3
## Radius limits, multiplied by the slime's current scale.
@export var min_radius_per_scale := 3.0
@export var max_radius_per_scale := 9.5

@export_group("Follow")
## Height of the look-at point above the slime's centre, per unit of slime scale.
@export var target_height_per_scale := 0.15
## Camera starts following once the slime is moving faster than this (0..1).
@export var follow_speed_threshold := 0.01

var target := Vector3.ZERO
var _dragging := false
var _last_scale := 1.0


func _ready() -> void:
    if follow:
        _last_scale = follow.slime_scale
        target = follow.position + Vector3.UP * target_height_per_scale * follow.slime_scale
        _update_camera()


func _unhandled_input(event: InputEvent) -> void:
    if follow == null:
        return
    if event.is_action_pressed("orbit_drag"):
        _dragging = true
    elif event.is_action_released("orbit_drag"):
        _dragging = false
    elif event is InputEventMouseMotion and _dragging:
        var motion := event as InputEventMouseMotion
        theta -= motion.relative.x * drag_sensitivity
        phi = clampf(phi - motion.relative.y * drag_sensitivity, min_phi, max_phi)
        _update_camera()
    elif event.is_action_pressed("zoom_in"):
        _zoom(-zoom_step)
    elif event.is_action_pressed("zoom_out"):
        _zoom(zoom_step)


func _zoom(amount: float) -> void:
    if follow == null:
        return
    var s := follow.slime_scale
    radius = clampf(radius + amount, min_radius_per_scale * s, max_radius_per_scale * s)
    _update_camera()


func _process(_delta: float) -> void:
    if follow == null or not is_instance_valid(follow):
        return
    var s := follow.slime_scale
    var target_look_y := follow.position.y + target_height_per_scale * s - (0.55 * s * follow.liquid_amt)
    if absf(s - _last_scale) > 1e-4:
        radius *= s / _last_scale
        target.y = target_look_y
        _last_scale = s
        _update_camera()
    else:
        target.y = lerpf(target.y, target_look_y, 0.1)
    target.x = follow.position.x
    target.z = follow.position.z
    _update_camera()


func _update_camera() -> void:
    var offset := Vector3(
        radius * sin(phi) * sin(theta),
        radius * cos(phi),
        radius * sin(phi) * cos(theta))
    var cam_pos := target + offset
    cam_pos.y = maxf(cam_pos.y, -1.2 + 0.25)
    camera.global_position = cam_pos
    camera.look_at(target, Vector3.UP)
```

FILE: scripts/hud.gd (89 lines)

```
class_name SlimeHud
extends Control
## HUD display for eaten stats, mass/volume, clone count, active status, and controls hints.

@export var slime: Slime
@export var props: SlimeProps
@export var hint_text := "WASD move · Shift run · Z hold liquid / engulf · E split follower · Q split roamer · M merge · Click poke"

var _stat: Label
var _hint: Label
var _last := ""


func _ready() -> void:
    set_anchors_preset(Control.PRESET_FULL_RECT)
    mouse_filter = Control.MOUSE_FILTER_IGNORE

    var row := HFlowContainer.new()
    row.mouse_filter = Control.MOUSE_FILTER_IGNORE
    row.add_theme_constant_override("h_separation", 10)
    row.add_theme_constant_override("v_separation", 6)
    row.set_anchors_preset(Control.PRESET_TOP_WIDE)
    row.offset_left = 12.0
    row.offset_top = 12.0
    row.offset_right = -12.0
    add_child(row)

    _stat = _make_pill(Color("eaf4fb"))
    row.add_child(_stat.get_parent())
    _hint = _make_pill(Color("bcd3e4"))
    _hint.text = hint_text
    row.add_child(_hint.get_parent())


func _make_pill(font_color: Color) -> Label:
    var panel := PanelContainer.new()
    panel.mouse_filter = Control.MOUSE_FILTER_IGNORE
    var box := StyleBoxFlat.new()
    box.bg_color = Color(16.0 / 255.0, 32.0 / 255.0, 48.0 / 255.0, 0.55)
    box.set_corner_radius_all(999)
    box.content_margin_left = 11.0
    box.content_margin_right = 11.0
    box.content_margin_top = 5.0
    box.content_margin_bottom = 5.0
    panel.add_theme_stylebox_override("panel", box)

    var label := Label.new()
    label.mouse_filter = Control.MOUSE_FILTER_IGNORE
    label.add_theme_font_size_override("font_size", 12)
    label.add_theme_color_override("font_color", font_color)
    panel.add_child(label)
    return label


func _process(_delta: float) -> void:
    if slime == null or props == null:
        return

    # Authoritative volume and mass calculations
    var mass := slime.get_volume()
    var clone_count := maxi(0, props.registered_slimes.size() - 1)
    var text := "Eaten %d / %d  ·  Vol %.1f m³ (Scale %.2f×)" % [props.eaten, props.total(), mass, slime.target_scale]
    if clone_count > 0:
        text += "  ·  Clones: %d" % clone_count

    # Check active push-eating engulf front or Z-cover progress
    var active_engulf_p = null
    for p in props.props:
        if p.last_presser == slime and p.engulf_front > 0.01:
            active_engulf_p = p
            break

    if active_engulf_p != null:
        var s_vol: float = slime.get_volume()
        var vr: float = active_engulf_p.real_volume / maxf(1e-4, s_vol)
        if vr >= 0.75:
            text += "  ·  Engulfing %d%% [Heavy Resistance]" % int(active_engulf_p.engulf_front * 100.0)
        elif vr >= 0.45:
            text += "  ·  Engulfing %d%% [Resisting]" % int(active_engulf_p.engulf_front * 100.0)
        else:
            text += "  ·  Engulfing %d%%" % int(active_engulf_p.engulf_front * 100.0)
    elif slime.engulf_progress > 0.05:
        text += "  ·  Z-Drape %d%%" % int(slime.engulf_progress * 100.0)
    elif slime.liquid_amt > 0.5:
        text += "  ·  [Liquid Mode]"

    if text != _last:
        _last = text
        _stat.text = text
```

FILE: shaders/slime.gdshader (625 lines)

```
shader_type spatial;
// Port of the Three.js "3D Anime Slime" vertex + fragment shaders.
// Colour maths is done in display (sRGB) space exactly like the HTML (which had no colour management),
// then converted to linear at the very end so Godot's output matches.
render_mode unshaded, depth_draw_always, fog_disabled, cull_back;

// ------------------------------------------------------------
// Inspector-editable look
// ------------------------------------------------------------
group_uniforms Shape;
uniform float uJiggleStrength : hint_range(0.0, 3.0, 0.01) = 1.0;

group_uniforms Refraction;
uniform float uDispersion : hint_range(0.0, 2.0, 0.01) = 0.85;
uniform float uRefractionStrength : hint_range(0.0, 1.0, 0.01) = 0.32;
uniform float uRippleNoiseScale = 1.65;
uniform float uBubbleNoiseScale = 3.5;

group_uniforms Surface;
uniform float uRimStrength : hint_range(0.0, 2.0, 0.01) = 1.15;
uniform float uOpacity : hint_range(0.0, 1.0, 0.01) = 0.48;
uniform float uNightMode : hint_range(0.0, 1.0, 0.01) = 0.0;
uniform vec3 uTopGlow = vec3(0.08, 0.18, 0.28);
uniform float uBubbleBoost = 1.4;

group_uniforms Colors;
uniform vec3 uDayShallow : source_color = vec3(0.52, 0.82, 0.98);
uniform vec3 uDayDeep : source_color = vec3(0.28, 0.62, 0.90);
uniform vec3 uNightShallow : source_color = vec3(0.22, 0.64, 0.94);
uniform vec3 uNightDeep : source_color = vec3(0.08, 0.32, 0.64);
uniform vec3 uRimDay : source_color = vec3(0.80, 0.96, 1.0);
uniform vec3 uRimNight : source_color = vec3(0.62, 0.92, 1.0);
uniform vec3 uOutlineDay : source_color = vec3(0.16, 0.34, 0.52);
uniform vec3 uOutlineNight : source_color = vec3(0.04, 0.12, 0.24);
uniform vec3 uCausticA : source_color = vec3(0.18, 0.92, 0.62);
uniform vec3 uCausticB : source_color = vec3(0.42, 0.98, 0.35);
uniform vec3 uBubbleColor : source_color = vec3(0.85, 1.0, 0.95);
uniform vec3 uEyeColor : source_color = vec3(1.0, 1.0, 1.0);

group_uniforms Eyes;
uniform float uEyeSeparation = 0.32;
uniform vec2 uEyeSize = vec2(0.11, 0.17);
uniform float uEyeElevation = 0.12;

group_uniforms;

// ------------------------------------------------------------
// Driven by Slime.gd every frame (not meant to be edited by hand)
// ------------------------------------------------------------
uniform float uMelt = 0.28; // set from Slime.melt (the script needs the same value for collision radii)
uniform float uImpulseTime = 10.0;
uniform vec3 uImpulsePos = vec3(0.0, 0.5, 1.0);
uniform vec3 uImpulseVector = vec3(0.0);
uniform float uImpulse2Time = 10.0;
uniform vec3 uImpulse2Pos = vec3(0.0, 0.5, 1.0);
uniform vec3 uImpulse2Vector = vec3(0.0);
uniform vec3 uMoveDir = vec3(0.0, 0.0, 1.0);
uniform float uMoveAmt = 0.0;
uniform float uCrawl = 0.0;
uniform vec3 uLean = vec3(0.0);
uniform vec3 uEatPos[6];
uniform vec3 uEatParam[6];
uniform vec3 uBoxHalf[6];
uniform vec3 uPressDir = vec3(0.0, 0.0, 1.0);
uniform float uPressAmt = 0.0;
uniform vec3 uSplatDir = vec3(0.0, 0.0, 1.0);
uniform float uSplat = 0.0;
uniform float uSpread = 0.0;
uniform float uWall = 2.15;
uniform vec3 uSolidPos[2];
uniform vec3 uSolidParam[2];
uniform vec3 uSolidHalf[2];
uniform vec3 uSolidDir[2];
```

```
uniform float uRippleIntensity = 0.12;
uniform float uFacing = 0.45;
uniform float uBlink = 1.0;
// Squeezing through a gap between solids: 0 = off, 1 = fully compressed across uSqueezeDir (a horizontal axis).
uniform float uSqueeze = 0.0;
uniform vec3 uSqueezeDir = vec3(1.0, 0.0, 0.0);

// Liquid puddle mode (hold Z key): flattens height, spreads outward, passes through small holes
uniform float uLiquid = 0.0;
uniform float uOverheadCompress = 1.0;

// Engulf drape mode (holding Z on top of a larger object)
uniform vec3 uDrapePos = vec3(0.0);
uniform vec3 uDrapeHalf = vec3(0.01);
uniform float uDrapeShape = 0.0;
uniform float uDrapeYaw = 0.0;
uniform float uDrapeAmt = 0.0;

// Budding fission (split animation for E and Q).
// uBudAmt 0→1: grows a secondary lobe out of the parent body.
// uBudDir: world-space direction the bud grows toward (normalized, local to this mesh).
// uBudScale: how large the bud lobe is relative to the parent body (0 → 1).
// Set uSplitAmt separately for the neck-snap oscillation impulse at detach moment.
uniform float uBudAmt = 0.0;
uniform vec3 uBudDir = vec3(0.0, 0.0, -1.0);
uniform float uBudScale = 0.0;
// Legacy: also keep uSplitAmt for the snap jiggle that fires on the clone side.
uniform float uSplitAmt = 0.0;
uniform vec3 uSplitDir = vec3(0.0, 0.0, -1.0);

uniform sampler2D tRefraction : hint_screen_texture, repeat_disable, filter_linear;

varying vec3 vWorldPosition;
varying vec3 vViewPosition;
varying vec3 vNormal;
varying vec3 vWorldNormal;
varying vec3 vLocalPos;
varying float vRippleMask;

// ---------------------------------------------------------------
// Vertex: deformation
// ---------------------------------------------------------------

// Regular n-gon in xz: x = distance along the nearest face normal (rho*cos), y = apothem. n<3 or n>=12 means round.
vec2 polyFold(vec2 xz, float n, float R) {
    float rho = length(xz);
    if (n < 2.5 || n >= 12.0) return vec2(rho, R);
    float sector = 6.28318531 / n;
    float ap = R * cos(3.14159265 / n);
    if (rho < 1e-5) return vec2(0.0, ap);
    float ang = atan(xz.x, xz.y);
    float a2 = mod(ang, sector) - 0.5 * sector;
    return vec2(rho * cos(a2), ap);
}

// shape = kind + sides*0.01  (kind: 0 box, 1 prism/cylinder, 2 pyramid/cone)
float sdShape(vec3 q, float shape, vec3 h) {
    float kind = floor(shape + 0.005);
    float n = floor((shape - kind) * 100.0 + 0.5);
    if (kind < 0.5) {
        vec3 a = abs(q) - h;
        return length(max(a, vec3(0.0))) + min(max(a.x, max(a.y, a.z)), 0.0);
    }
    vec2 f = polyFold(q.xz, n, h.x);
    if (kind < 1.5) {
        vec2 d2 = vec2(f.x - f.y, abs(q.y) - h.y);
        return min(max(d2.x, d2.y), 0.0) + length(max(d2, vec2(0.0)));
    }
    float r = f.y;
    float H2 = 2.0 * h.y;
    float dSide = (f.x * H2 + (q.y - h.y) * r) / sqrt(H2 * H2 + r * r);
    return max(dSide, -q.y - h.y);
}

// Push depth along -n that lifts a point to the offset surface: exact exit distance, iterated (stays correct for deep dents)
float pushAmt(vec3 q, vec3 nL, float shape, vec3 h, float e) {
    float s = 0.0;
    for (int k = 0; k < 4; k++) {
        s += max(e - sdShape(q - nL * s, shape, h), 0.0);
    }
    return s;
}

// The depth field is blurred over a small disc of taps in the plane facing the prop, so the dent wall at the
// prop's silhouette becomes a smooth ramp instead of a cliff the mesh can't resolve.
vec3 dentPush(vec3 p, vec3 c, float shape, vec3 h, float yaw, float blur, vec3 nOv) {
    float cy = cos(yaw);
    float sy = sin(yaw);
    vec3 dv = p - c;
    vec3 q = vec3(dv.x * cy - dv.z * sy, dv.y, dv.x * sy + dv.z * cy);
    float e = 0.08;
    if (sdShape(q, shape, h) > e + blur * 1.2) return vec3(0.0);
    vec3 n = dot(nOv, nOv) > 0.5 ? nOv : normalize(vec3(c.x, 0.0, c.z) + vec3(0.0001, 0.0, 0.0001));
    vec3 nL = vec3(n.x * cy - n.z * sy, 0.0, n.x * sy + n.z * cy);
    vec3 u = vec3(-nL.z, 0.0, nL.x);
    vec3 v = vec3(0.0, 1.0, 0.0);
    float R = blur;
    float acc = pushAmt(q, nL, shape, h, e) * 0.28;
    acc += pushAmt(q + u * R, nL, shape, h, e) * 0.12;
    acc += pushAmt(q - u * R, nL, shape, h, e) * 0.12;
    acc += pushAmt(q + v * R, nL, shape, h, e) * 0.12;
    acc += pushAmt(q - v * R, nL, shape, h, e) * 0.12;
    acc += pushAmt(q + (u + v) * R * 0.7071, nL, shape, h, e) * 0.06;
    acc += pushAmt(q + (u - v) * R * 0.7071, nL, shape, h, e) * 0.06;
    acc += pushAmt(q + (-u + v) * R * 0.7071, nL, shape, h, e) * 0.06;
    acc += pushAmt(q + (-u - v) * R * 0.7071, nL, shape, h, e) * 0.06;
    return -n * acc;
}

// ---------------------------------------------------------------
// Solid-contact dent (replaces dentPush for obstacles the slime is squashed against)
//
// dentPush shoves every point along ONE fixed direction (the contact normal) until it clears the obstacle. That is fine
// for a flat face, but against a corner it pinches the skin into a kinked notch (points on the two flanks get dragged
// diagonally, the tip gets thrown back along the axis). This version instead pushes each point along the obstacle's own
// outward normal at that point, by the distance it is inside the offset outline, so the skin follows the real outline
// and the push fades out continuously (no cliff). Corners of boxes are rounded a little so the jelly wraps round them.
// ---------------------------------------------------------------
const float SOLID_OFFSET = 0.08;  // skin gap between jelly and obstacle
const float SOLID_ROUND = 0.25;   // corner rounding radius (flat faces are unaffected)

// Footprint (xz) signed distance to the obstacle and its outward normal there: returns vec3(sd, nx, nz).
vec3 footSd(vec3 q, float shape, vec3 h, float rr) {
    float kind = floor(shape + 0.005);
    float n = floor((shape - kind) * 100.0 + 0.5);
    if (kind < 0.5) {
        vec2 hs = max(h.xz - vec2(rr), vec2(0.01));
        vec2 a = abs(q.xz) - hs;
        vec2 d = q.xz - clamp(q.xz, -hs, hs);
        float od = length(d);
        float sd = od + min(max(a.x, a.y), 0.0) - rr;
        vec2 nn;
        if (a.x > 0.0 && a.y > 0.0) {
            nn = d / max(od, 1e-5);
        } else if (a.x > a.y) {
            nn = vec2(q.x >= 0.0 ? 1.0 : -1.0, 0.0);
        } else {
            nn = vec2(0.0, q.z >= 0.0 ? 1.0 : -1.0);
        }
        return vec3(sd, nn);
    }
    // prism (kind 1) keeps its radius; pyramid/cone (kind 2) narrows with height, like the collision reach does
    float R = h.x;
    if (kind > 1.5) R *= clamp(1.0 - (q.y + h.y) / (2.0 * h.y), 0.06, 1.0);
    float rho = length(q.xz);
    if (n < 2.5 || n >= 12.0) {
        return vec3(rho - R, q.xz / max(rho, 1e-5));
    }
    float sector = 6.28318531 / n;
    float faceAng = (floor(atan(q.x, q.z) / sector) + 0.5) * sector;
    vec2 nf = vec2(sin(faceAng), cos(faceAng));
    return vec3(dot(q.xz, nf) - R * cos(3.14159265 / n), nf);
}

// Horizontal push at one local-space point: along the outward normal, by how far the point is inside the offset outline.
// Fades out over the obstacle's top edge so the jelly above a low obstacle is left alone.
vec3 solidPushLocal(vec3 q, float shape, vec3 h) {
    vec3 f = footSd(q, shape, h, SOLID_ROUND);
    float win = 1.0 - smoothstep(0.0, 0.2, q.y - h.y);
    float mag = max(SOLID_OFFSET - f.x, 0.0) * win;
    return vec3(f.y * mag, 0.0, f.z * mag);
}

// Same tap pattern as dentPush (a small disc of taps facing the obstacle) so the push is averaged, not aliased.
vec3 solidDent(vec3 p, vec3 c, float shape, vec3 h, float yaw, float blur, vec3 nOv) {
    float cy = cos(yaw);
    float sy = sin(yaw);
    vec3 dv = p - c;
    vec3 q = vec3(dv.x * cy - dv.z * sy, dv.y, dv.x * sy + dv.z * cy);
    vec3 nL = vec3(nOv.x * cy - nOv.z * sy, 0.0, nOv.x * sy + nOv.z * cy);
    vec3 u = vec3(-nL.z, 0.0, nL.x);
    vec3 v = vec3(0.0, 1.0, 0.0);
    float R = blur;
    vec3 acc = solidPushLocal(q, shape, h) * 0.28;
    acc += solidPushLocal(q + u * R, shape, h) * 0.12;
    acc += solidPushLocal(q - u * R, shape, h) * 0.12;
    acc += solidPushLocal(q + v * R, shape, h) * 0.12;
    acc += solidPushLocal(q - v * R, shape, h) * 0.12;
    acc += solidPushLocal(q + (u + v) * R * 0.7071, shape, h) * 0.06;
    acc += solidPushLocal(q + (u - v) * R * 0.7071, shape, h) * 0.06;
    acc += solidPushLocal(q + (-u + v) * R * 0.7071, shape, h) * 0.06;
    acc += solidPushLocal(q + (-u - v) * R * 0.7071, shape, h) * 0.06;
    return vec3(acc.x * cy + acc.z * sy, acc.y, -acc.x * sy + acc.z * cy);
}

vec3 deformSlime(vec3 pos, vec3 norm, float t) {
    vec3 p = pos;

    float height01 = clamp((p.y / 1.6 + 1.0) * 0.5, 0.0, 1.0);
    float bellyCurve = sin(height01 * 3.14159265);
    float lowerBulge = pow(1.0 - height01, 1.35) * smoothstep(0.0, 0.22, height01);
    float bellySpread = 1.0 + (lowerBulge * 0.34 + bellyCurve * 0.08) * (0.8 + uMelt * 1.1);
    p.xz *= bellySpread;

    p.y *= (0.86 - uMelt * 0.22);

    float groundFloor = -1.6 * (0.86 - uMelt * 0.22);

    float baseH = p.y - groundFloor;
    if (baseH < 0.9) {
        float tB = clamp(baseH / 0.9, 0.0, 1.0);
        p.y = groundFloor + 0.9 * (2.0 * tB * tB - tB * tB);
    }

    if (uSqueeze > 0.001) {
        // Squeezing through a gap: bottleneck constriction along flow passage (neck pinched, front/rear bulges)
        float sq = clamp(uSqueeze, 0.0, 0.94);
        vec2 sqd = normalize(uSqueezeDir.xz + vec2(0.0001));
        vec2 fwdFlow = vec2(-sqd.y, sqd.x);
        float sqa = dot(p.xz, sqd);
        float flowDist = dot(p.xz, fwdFlow);
        // Local neck constriction: maximum compression at neck (flowDist ~ 0), relaxing towards front and rear
        float neckProfile = exp(-flowDist * flowDist * 1.6);
        float localSq = sq * mix(0.4, 1.0, neckProfile);
        float widthScale = max(0.08, 1.0 - localSq);
        // Front and rear bulge out to conserve fluid volume
        float bulge = sq * (1.0 - neckProfile) * 0.45;
        float lengthScale = 1.0 + sq * 0.75 + bulge;
        float heightScale = 1.0 + localSq * 0.35 * (1.0 - uLiquid);
        p.xz = sqd * (sqa * widthScale) + fwdFlow * (flowDist * lengthScale);
        p.y = groundFloor + (p.y - groundFloor) * heightScale;
    }

    if (uLiquid > 0.001) {
        // Volume-preserving liquid puddle flattening:
        // At 22% height, conservation of volume requires spread = 1 / sqrt(0.22) = 2.13x.
        float targetH = 0.22 * uOverheadCompress;
        float liqH = mix(1.0, targetH, uLiquid);
        float liqR = mix(1.0, 1.0 / sqrt(max(0.04, targetH)), uLiquid);
        if (uSqueeze > 0.001) {
            vec2 sqd = normalize(uSqueezeDir.xz + vec2(0.0001));
            vec2 fwdFlow = vec2(-sqd.y, sqd.x);
            float sqa = dot(p.xz, sqd);
            float flowDist = dot(p.xz, fwdFlow);
            // Area-preserving squeeze in liquid: compressing sqd axis expands fwdFlow proportionally
            p.xz = sqd * (sqa * mix(1.0, 0.45, uLiquid)) + fwdFlow * (flowDist * mix(1.0, liqR * 1.25, uLiquid));
        } else {
            p.xz *= liqR;
        }
        p.y = groundFloor + (p.y - groundFloor) * liqH;

        // Directional flow surge: the leading edge surges forward in movement direction
        vec2 mdir = normalize(uMoveDir.xz + vec2(0.0001));
        float flowAlong = dot(p.xz, mdir);
        p.xz += mdir * (uMoveAmt * 0.30 * uLiquid * smoothstep(-1.2, 1.8, flowAlong));

        // Viscous amoebic perimeter ripples (surface tension lobes)
        float ang = atan(p.z, p.x);
        float edgeWobble = (sin(ang * 4.0 + t * 4.0) * 0.06 + sin(ang * 6.0 - t * 3.0) * 0.03) * uLiquid;
        float distCenter = length(p.xz);
        p.xz *= 1.0 + edgeWobble * smoothstep(0.4, 2.0, distCenter);

        // Capillary surface ripples propagating across the liquid sheet
        float capRipple = sin(distCenter * 4.0 - t * 7.0 + flowAlong * 2.0) * 0.030 * uLiquid;
        p.y += capRipple * smoothstep(0.0, 0.35, p.y - groundFloor);
    }

    // ---- Sticky Droplet Budding (E and Q split) ----
    // Softly draws out a sticky droplet peak toward the separating partner.
    // Preserves smooth convex jelly curvature with NO flat cut face, NO pinch dents, and NO distorted peanut mesh.
    if (uSplitAmt > 0.001) {
        vec2 sdir = normalize(uSplitDir.xz + vec2(0.0001));
        float sproj = dot(p.xz, sdir);

        // Soft teardrop stretch toward the separating partner
        float stretch = smoothstep(-0.3, 1.2, sproj) * uSplitAmt * 0.35;

        p.xz += sdir * stretch;
    }

    float breathe = sin(t * 2.0) * 0.022 * uJiggleStrength;
    p.y *= (1.0 + breathe);
    p.xz *= (1.0 - breathe * 0.45);

    float gentleSway = sin(p.y * 2.2 - t * 2.4) * 0.012 * uJiggleStrength;
```

```
    p.xz += norm.xz * gentleSway * smoothstep(0.05, 0.5, height01);

    float distToPoke = distance(pos, uImpulsePos);
    float pokeInfluence = exp(-distToPoke * 1.15);
    float springOsc = sin(uImpulseTime * 14.0) * exp(-uImpulseTime * 3.0);
    vec3 impulseOffset = uImpulseVector * pokeInfluence * springOsc * uJiggleStrength;

    float distToPoke2 = distance(pos, uImpulse2Pos);
    float pokeInfluence2 = exp(-distToPoke2 * 1.15);
    float springOsc2 = sin(uImpulse2Time * 14.0) * exp(-uImpulse2Time * 3.0);
    vec3 impulseOffset2 = uImpulse2Vector * pokeInfluence2 * springOsc2 * uJiggleStrength;

    float globalBounce = sin(uImpulseTime * 11.0) * exp(-uImpulseTime * 2.6) * length(uImpulseVector) * 0.32 * uJiggleStrength;
    p.y += globalBounce * (height01 - 0.15);
    p.xz -= globalBounce * 0.38 * p.xz * (1.0 - height01 * 0.5);

    p += (impulseOffset + impulseOffset2) * smoothstep(0.02, 0.25, height01);

    for (int i = 0; i < 6; i++) {
        if (uEatParam[i].y > 0.001) {
            p += dentPush(p, uEatPos[i], uEatParam[i].x, uBoxHalf[i], uEatParam[i].z, 0.26, vec3(0.0)) * uEatParam[i].y;
        }
    }
    // In liquid puddle mode the slime is a flat disk — press/splat/solid-dent deformations
    // were designed for the upright blob and look wrong on the expanded puddle. Fade them out.
    float solidBlend = 1.0 - uLiquid;

    float effPress = uPressAmt * solidBlend;
    if (effPress > 0.001) {
        vec2 pd2 = normalize(uPressDir.xz + vec2(0.0001));
        float al = dot(p.xz, pd2);
        vec2 pp = p.xz - pd2 * al;
        float anchor = 1.9;
        al = anchor + (al - anchor) * (1.0 - 0.12 * effPress);
        p.xz = pd2 * al + pp * (1.0 + 0.07 * effPress);
        p.y = groundFloor + (p.y - groundFloor) * (1.0 + 0.05 * effPress);
    }
    float effSplat = uSplat * solidBlend;
    float effSpread = uSpread * solidBlend;
    if (effSplat > 0.001 || effSpread > 0.001) {
        // Splat model: the body compresses toward the wall (anchored at the contact plane), the top carries on forward
        // while the floor-gripping base lags, and the mass spreads sideways/up with a slower lag (viscous).
        vec2 sd = normalize(uSplatDir.xz + vec2(0.0001));
        float sa = dot(p.xz, sd);
        vec2 sperp = p.xz - sd * sa;
        float wallS = uWall;
        sa = wallS + (sa - wallS) * (1.0 - 0.34 * effSplat);
        sa += effSplat * 0.16 * (height01 - 0.15);
        p.xz = sd * sa + sperp * (1.0 + 0.34 * effSpread);
        p.y = groundFloor + (p.y - groundFloor) * (1.0 + 0.10 * effSpread);
    }
    p.y = max(p.y, groundFloor);

    float wave = 0.5 + 0.5 * sin(uCrawl);
    float sy = 1.0 + uMoveAmt * (0.10 * wave - 0.04);
    p.xz *= 1.0 / sqrt(sy);
    p.y = groundFloor + (p.y - groundFloor) * sy;
    vec2 d = normalize(uMoveDir.xz + vec2(0.0001));
    p.xz -= uLean.xz * (height01 - 0.2) * 0.38;
    float tw = sin(uCrawl - dot(pos.xz, d) * 2.8);
    float twMask = smoothstep(0.1, 0.6, height01);
    p.y += uMoveAmt * 0.07 * tw * twMask;
    p.xz += norm.xz * uMoveAmt * 0.03 * tw * twMask;

    // Wrap the body around whatever it is squashed against — faded out in liquid mode or heavy squeeze.
    float dentBlend = solidBlend * (1.0 - clamp(uSqueeze, 0.0, 1.0) * 0.65);
    for (int i = 0; i < 2; i++) {
        float solidAmt = uSolidParam[i].y * dentBlend;
        if (solidAmt > 0.001) {
            float ring = length(solidDent(p, uSolidPos[i], uSolidParam[i].x, uSolidHalf[i], uSolidParam[i].z, 0.55, uSolidDir[i]));
            p += norm * ring * 0.3 * solidAmt;
            p += solidDent(p, uSolidPos[i], uSolidParam[i].x, uSolidHalf[i], uSolidParam[i].z, 0.15, uSolidDir[i]) * solidAmt;
        }
    }

    if (uDrapeAmt > 0.001) {
        float cy = cos(uDrapeYaw);
        float sy = sin(uDrapeYaw);
        vec3 dv = p - uDrapePos;
        vec3 q = vec3(dv.x * cy - dv.z * sy, dv.y, dv.x * sy + dv.z * cy);
        vec2 topReach = max(uDrapeHalf.xz * 1.18, vec2(0.2));
        p.xz = mix(p.xz, normalize(p.xz + vec2(0.0001)) * length(topReach), uDrapeAmt * 0.65);
        if (dv.y < 0.0) {
            p.y = mix(p.y, uDrapePos.y - abs(dv.y) * 1.25, uDrapeAmt * 0.45);
        }
    }

    return p;
}

void vertex() {
    vec3 position = VERTEX;
    vec3 nrm = NORMAL;
    vLocalPos = position;

    vec3 tangent = normalize(cross(nrm, vec3(0.0, 1.0, 0.1)));
    vec3 bitangent = normalize(cross(nrm, tangent));
    float eps = 0.025;

    vec3 p0 = deformSlime(position, nrm, TIME);
    vec3 p1 = deformSlime(position + tangent * eps, normalize(position + tangent * eps), TIME);
    vec3 p2 = deformSlime(position + bitangent * eps, normalize(position + bitangent * eps), TIME);

    vec3 deformedNormal = normalize(cross(p1 - p0, p2 - p0));

    vec4 worldPos = MODEL_MATRIX * vec4(p0, 1.0);
    vWorldPosition = worldPos.xyz;

    vec4 mvPosition = VIEW_MATRIX * worldPos;
    vViewPosition = -mvPosition.xyz;

    vNormal = normalize(mat3(MODELVIEW_MATRIX) * deformedNormal);
    vWorldNormal = normalize(mat3(MODEL_MATRIX) * deformedNormal);

    float distToPoke = distance(position, uImpulsePos);
    float distToPoke2 = distance(position, uImpulse2Pos);
    vRippleMask = exp(-distToPoke * 1.5) * exp(-uImpulseTime * 2.5) + exp(-distToPoke2 * 1.5) * exp(-uImpulse2Time * 2.5);

    VERTEX = p0;
    NORMAL = deformedNormal;
}

// ---------------------------------------------------------
// Fragment
// ---------------------------------------------------------

vec3 srgb2lin(vec3 c) {
    vec3 hi = pow((c + vec3(0.055)) / 1.055, vec3(2.4));
    vec3 lo = c / 12.92;
    return mix(hi, lo, step(c, vec3(0.04045)));
}

vec3 lin2srgb(vec3 c) {
    c = max(c, vec3(0.0));
    vec3 hi = 1.055 * pow(c, vec3(1.0 / 2.4)) - vec3(0.055);
    vec3 lo = c * 12.92;
    return mix(hi, lo, step(c, vec3(0.0031308)));
}

vec4 permute(vec4 x) { return mod(((x * 34.0) + 1.0) * x, 289.0); }
vec4 taylorInvSqrt(vec4 r) { return 1.79284291400159 - 0.85373472095314 * r; }

float snoise(vec3 v) {
    const vec2 C = vec2(1.0 / 6.0, 1.0 / 3.0);
    const vec4 D = vec4(0.0, 0.5, 1.0, 2.0);
    vec3 i = floor(v + dot(v, C.yyy));
    vec3 x0 = v - i + dot(i, C.xxx);
    vec3 g = step(x0.yzx, x0.xyz);
    vec3 l = 1.0 - g;
    vec3 i1 = min(g.xyz, l.zxy);
    vec3 i2 = max(g.xyz, l.zxy);
    vec3 x1 = x0 - i1 + 1.0 * C.xxx;
    vec3 x2 = x0 - i2 + 2.0 * C.xxx;
    vec3 x3 = x0 - 1.0 + 3.0 * C.xxx;
    i = mod(i, 289.0);
    vec4 p = permute(permute(permute(
             i.z + vec4(0.0, i1.z, i2.z, 1.0))
           + i.y + vec4(0.0, i1.y, i2.y, 1.0))
           + i.x + vec4(0.0, i1.x, i2.x, 1.0));
    float n_ = 1.0 / 7.0;
    vec3 ns = n_ * D.wyz - D.xzx;
    vec4 j = p - 49.0 * floor(p * ns.z * ns.z);
    vec4 x_ = floor(j * ns.z);
    vec4 y_ = floor(j - 7.0 * x_);
    vec4 x = x_ * ns.x + ns.yyyy;
    vec4 y = y_ * ns.x + ns.yyyy;
    vec4 h = 1.0 - abs(x) - abs(y);
    vec4 b0 = vec4(x.xy, y.xy);
    vec4 b1 = vec4(x.zw, y.zw);
    vec4 s0 = floor(b0) * 2.0 + 1.0;
    vec4 s1 = floor(b1) * 2.0 + 1.0;
    vec4 sh = -step(h, vec4(0.0));
    vec4 a0 = b0.xzyw + s0.xzyw * sh.xxyy;
    vec4 a1 = b1.xzyw + s1.xzyw * sh.zzww;
    vec3 p0 = vec3(a0.xy, h.x);
    vec3 p1 = vec3(a0.zw, h.y);
    vec3 p2 = vec3(a1.xy, h.z);
    vec3 p3 = vec3(a1.zw, h.w);
    vec4 nrm = taylorInvSqrt(vec4(dot(p0, p0), dot(p1, p1), dot(p2, p2), dot(p3, p3)));
    p0 *= nrm.x;
    p1 *= nrm.y;
    p2 *= nrm.z;
    p3 *= nrm.w;
    vec4 m = max(0.6 - vec4(dot(x0, x0), dot(x1, x1), dot(x2, x2), dot(x3, x3)), 0.0);
    m = m * m;
    return 42.0 * dot(m * m, vec4(dot(p0, x0), dot(p1, x1), dot(p2, x2), dot(p3, x3)));
}

// Screen texture is linear in Godot; the HTML worked on display values, so convert on the way in.
vec3 sampleScreen(vec2 uvGl) {
    return lin2srgb(texture(tRefraction, vec2(uvGl.x, 1.0 - uvGl.y)).rgb);
}

void fragment() {
    float t = TIME;
    // GL-style screen UV (origin bottom-left) so every offset below keeps the HTML's sign convention.
    vec2 screenUV = vec2(SCREEN_UV.x, 1.0 - SCREEN_UV.y);
    vec3 viewDir = normalize(vViewPosition);
    vec3 normal = normalize(vNormal);

    vec3 worldNormal = normalize(vWorldNormal);

    vec3 noiseCoord = vWorldPosition * uRippleNoiseScale + vec3(0.0, -t * 0.45, t * 0.25);
    float n1 = snoise(noiseCoord);
    float n2 = snoise(noiseCoord * 1.8 - vec3(t * 0.35));

    float ndotv = max(0.0, dot(normal, viewDir));
    float edgeFactor = pow(1.0 - ndotv, 1.5);

    vec2 rippleOffset = vec2(n1 * 0.7 + n2 * 0.3, n2 * 0.7 - n1 * 0.3)
                      * 0.012 * uRippleIntensity * (0.35 + 0.65 * edgeFactor + vRippleMask * 1.2);

    vec2 lensOffset = normal.xy * (0.065 * uRefractionStrength);
    vec2 totalDistortion = lensOffset + rippleOffset;

    float disp = 0.009 * uDispersion;
    vec2 dir = normalize(totalDistortion + vec2(0.0001));

    vec2 uvR = clamp(screenUV - totalDistortion * (1.0 - 0.18 * uDispersion) + dir * disp * 0.4, vec2(0.002), vec2(0.998));
    vec2 uvG = clamp(screenUV - totalDistortion * 1.0, vec2(0.002), vec2(0.998));
    vec2 uvGY = clamp(screenUV - totalDistortion * (1.0 + 0.10 * uDispersion) - dir * disp * 0.25, vec2(0.002), vec2(0.998));
    vec2 uvB = clamp(screenUV - totalDistortion * (1.0 + 0.22 * uDispersion) - dir * disp * 0.55, vec2(0.002), vec2(0.998));

    vec3 sR = sampleScreen(uvR);
    vec3 sG = sampleScreen(uvG);
    vec3 sGY = sampleScreen(uvGY);
    vec3 sB = sampleScreen(uvB);

    vec3 refractedColor = vec3(sR.r, mix(sG.g, sGY.g, 0.5), sB.b);

    float edgeDelta = clamp(length(sG - sB) * 1.5, 0.0, 0.45);
    vec3 greenCyanCaustic = mix(lin2srgb(uCausticA), lin2srgb(uCausticB), n1 * 0.5 + 0.5);
    float basalMask = smoothstep(0.05, -0.72, vLocalPos.y) * smoothstep(0.1, 0.85, 1.0 - ndotv);
    refractedColor += greenCyanCaustic * edgeDelta * 0.45 * uDispersion;
    refractedColor = mix(refractedColor, greenCyanCaustic * 0.92, basalMask * 0.28 * clamp(uDispersion, 0.0, 1.4));

    float fresnel = pow(1.0 - ndotv, 2.6);

    vec3 shallowTint = mix(lin2srgb(uDayShallow), lin2srgb(uNightShallow), uNightMode);
    vec3 deepTint = mix(lin2srgb(uDayDeep), lin2srgb(uNightDeep), uNightMode);

    float thickness = pow(ndotv, 0.65);
    vec3 jellyTint = mix(shallowTint, deepTint, thickness);

    float effectiveOpacity = clamp(uOpacity * (0.72 + 0.48 * thickness), 0.05, 0.95);
    vec3 color = mix(refractedColor, jellyTint, effectiveOpacity);

    float topGradient = smoothstep(-0.8, 1.0, worldNormal.y);
    color += uTopGlow * topGradient * (1.0 - uNightMode * 0.4);

    float innerRim = smoothstep(0.38, 0.82, fresnel) * uRimStrength;
    vec3 rimColor = mix(lin2srgb(uRimDay), lin2srgb(uRimNight), uNightMode);
    color = mix(color, rimColor, clamp(innerRim * 0.78, 0.0, 0.92));

    vec3 localP = vLocalPos * uBubbleNoiseScale + vec3(0.0, t * 0.6, 0.0);
    float bubbleNoise1 = snoise(localP);
    float bubbleNoise2 = snoise(localP * 2.2 - vec3(0.0, t * 1.1, 0.0));
    float bubbleField = smoothstep(0.55, 0.72, bubbleNoise1) * smoothstep(0.4, 0.6, bubbleNoise2);
    float depthFactor = smoothstep(-0.8, 0.6, vLocalPos.y);
    vec3 bubbleColor = lin2srgb(uBubbleColor) * uBubbleBoost;
    color = mix(color, bubbleColor, bubbleField * depthFactor * 0.55 * clamp(uRippleIntensity * 2.0, 0.2, 1.5));

    vec3 lp = normalize(vLocalPos);
    float eyeYaw = atan(lp.x, lp.z) - uFacing;
    eyeYaw = atan(sin(eyeYaw), cos(eyeYaw));
    float eyeElev = asin(clamp(lp.y, -1.0, 1.0));
    float targetElev = mix(uEyeElevation, 0.48, uLiquid);
    vec2 eyeSz = mix(uEyeSize, vec2(uEyeSize.x * 1.3, uEyeSize.y * 0.9), uLiquid);
    vec2 eq = vec2((abs(eyeYaw) - uEyeSeparation) / eyeSz.x, (eyeElev - targetElev) / (eyeSz.y * uBlink));
    float eyeMask = 1.0 - smoothstep(0.82, 1.0, length(eq));
    color = mix(color, lin2srgb(uEyeColor), eyeMask * 0.97);

    float celOutline = smoothstep(0.84, 0.95, 1.0 - ndotv);
    vec3 outlineColor = mix(lin2srgb(uOutlineDay), lin2srgb(uOutlineNight), uNightMode);
    color = mix(color, outlineColor, celOutline * 0.55 * clamp(uRimStrength, 0.0, 1.5));

    ALBEDO = srgb2lin(clamp(color, vec3(0.0), vec3(1.0));
}
```

FILE: shaders/floor.gdshader (157 lines)

```
shader_type spatial;
// Studio floor. Two looks, switched by uDevTexture:
//  - on : prototype / "early development" floor (checker cells + minor lines + bold major lines, in world units)
//  - off: the HTML's original thin-line tiled grid
// Both use Three-style lighting (ambient + one directional, computed in display space), linear fog, and the blob shadow.
// The shadow lives in here (not a separate quad) so it ends up in the screen texture the slime refracts.
render_mode unshaded, fog_disabled;

group_uniforms Style;
uniform bool uDevTexture = true;

group_uniforms DevTexture;
uniform float uCellSize = 1.0;              // world units per checker cell
uniform float uMajorEvery = 4.0;            // a bold line every N cells
uniform vec3 uLightCell : source_color = vec3(0.46, 0.50, 0.54);
uniform vec3 uDarkCell : source_color = vec3(0.37, 0.41, 0.45);
uniform vec3 uMinorLine : source_color = vec3(0.30, 0.34, 0.38);
uniform vec3 uMajorLine : source_color = vec3(0.85, 0.91, 0.97);
uniform float uMinorWidth = 0.025;          // half-width, in cells
uniform float uMajorWidth = 0.02;           // half-width, in major cells
uniform float uMinorAlpha : hint_range(0.0, 1.0, 0.01) = 0.6;
uniform float uMajorAlpha : hint_range(0.0, 1.0, 0.01) = 0.85;
uniform float uFadeStart = 0.35;            // detail fades out when a cell shrinks below this many... (pixels^-1), kills moire in the distance
uniform float uFadeEnd = 0.8;

group_uniforms OriginalGrid;
uniform float uTiles = 100.0;
uniform float uPlaneSize = 400.0;           // size of the floor plane mesh, used to rebuild UVs from world position
uniform vec3 uBaseColor : source_color = vec3(0.435, 0.490, 0.549);
uniform vec3 uEdgeColor : source_color = vec3(0.824, 0.910, 0.980);
uniform float uEdgeAlpha : hint_range(0.0, 1.0, 0.01) = 0.55;
uniform float uMidAlpha : hint_range(0.0, 1.0, 0.01) = 0.18;
uniform float uEdgeHalfWidth = 0.00390625; // 2px of a 512px tile, drawn on each side of a tile seam
uniform float uMidHalfWidth = 0.001953125; // 1px of a 512px tile

group_uniforms Lighting;
uniform vec3 uAmbientColor : source_color = vec3(0.859, 0.918, 0.996);
uniform float uAmbientEnergy = 0.85;
uniform vec3 uLightColor : source_color = vec3(1.0, 1.0, 1.0);
uniform float uLightEnergy = 0.9;
uniform vec3 uLightDir = vec3(4.0, 8.0, 5.0); // direction TOWARD the light (keep in sync with the DirectionalLight3D)

group_uniforms Fog;
uniform vec3 uFogColor : source_color = vec3(0.337, 0.471, 0.565);
uniform float uFogNear = 25.0;
uniform float uFogFar = 90.0;

group_uniforms BlobShadow;
uniform vec3 uShadowColor : source_color = vec3(0.039, 0.078, 0.137);
uniform float uShadowInnerAlpha : hint_range(0.0, 1.0, 0.01) = 0.6;
uniform float uShadowMidAlpha : hint_range(0.0, 1.0, 0.01) = 0.35;
uniform float uShadowMidStop = 0.7;
uniform float uShadowInnerStop = 0.0625; // 4px of a 64px radius
uniform float uShadowRadius = 2.0;       // half-size of the HTML's 4x4 quad, at slime scale 1

group_uniforms;

// Driven by Slime.gd
uniform int uBlobCount = 1;
uniform vec3 uBlobPos[8];
uniform float uBlobScale[8];

varying vec3 vWorld;

vec3 srgb2lin(vec3 c) {
    vec3 hi = pow((c + vec3(0.055)) / 1.055, vec3(2.4));
    vec3 lo = c / 12.92;
    return mix(hi, lo, step(c, vec3(0.04045)));
}

vec3 lin2srgb(vec3 c) {
    c = max(c, vec3(0.0));
    vec3 hi = 1.055 * pow(c, vec3(1.0 / 2.4)) - vec3(0.055);
    vec3 lo = c * 12.92;
    return mix(hi, lo, step(c, vec3(0.0031308)));
}

// 1 on a line of half-width hw, centred on integer coordinates of w (anti-aliased with the pixel footprint).
float gridLine(vec2 w, float hw) {
    vec2 f = fract(w);
    vec2 fw = fwidth(w);
    float aa = max(max(fw.x, fw.y) * 0.75, 1e-5);
    float seam = min(min(f.x, 1.0 - f.x), min(f.y, 1.0 - f.y));
    return 1.0 - smoothstep(hw - aa, hw + aa, seam);
}

vec3 devAlbedo(vec2 worldXZ) {
    vec2 w = worldXZ / uCellSize;
    vec2 cell = floor(w);
    float checker = mod(cell.x + cell.y, 2.0);
    vec3 lightC = lin2srgb(uLightCell);
    vec3 darkC = lin2srgb(uDarkCell);
    vec3 col = mix(lightC, darkC, checker);

    float minor = gridLine(w, uMinorWidth);
    float major = gridLine(w / uMajorEvery, uMajorWidth);
    col = mix(col, lin2srgb(uMinorLine), minor * uMinorAlpha);
    col = mix(col, lin2srgb(uMajorLine), major * uMajorAlpha);

    // In the distance cells get sub-pixel: blend toward the average colour so it doesn't shimmer.
    vec2 fw = fwidth(w);
    float fade = smoothstep(uFadeStart, uFadeEnd, max(fw.x, fw.y));
    vec3 avg = mix(lightC, darkC, 0.5);
    return mix(col, avg, fade);
}

vec3 originalAlbedo(vec2 uv) {
    vec2 f = fract(uv * uTiles);
    vec2 fw = fwidth(uv * uTiles);
    float aa = max(fw.x, fw.y) * 0.75;
    float seam = min(min(f.x, 1.0 - f.x), min(f.y, 1.0 - f.y));
    float edge = 1.0 - smoothstep(uEdgeHalfWidth - aa, uEdgeHalfWidth + aa, seam);
    float midd = min(abs(f.x - 0.5), abs(f.y - 0.5));
    float mid = 1.0 - smoothstep(uMidHalfWidth - aa, uMidHalfWidth + aa, midd);

    vec3 base = lin2srgb(uBaseColor);
    vec3 edgeC = lin2srgb(uEdgeColor);
    vec3 albedo = mix(base, edgeC, mid * uMidAlpha);
    return mix(albedo, edgeC, edge * uEdgeAlpha);
}

void vertex() {
    vWorld = (MODEL_MATRIX * vec4(VERTEX, 1.0)).xyz;
}

void fragment() {
    vec3 devCol = devAlbedo(vWorld.xz);
    vec3 origCol = originalAlbedo(vWorld.xz / uPlaneSize + vec2(0.5));
    vec3 albedo = uDevTexture ? devCol : origCol;

    // --- Three.js-style lighting on an up-facing surface, display space ---
    float ndl = max(normalize(uLightDir).y, 0.0);
    vec3 lit = albedo * (lin2srgb(uAmbientColor) * uAmbientEnergy + lin2srgb(uLightColor) * uLightEnergy * ndl);
    vec3 color = clamp(lit, vec3(0.0), vec3(1.0));

    // --- blob shadow (canvas radial gradient: inner circle r=4px, outer r=64px, up to 8 slimes) ---
    float max_shadow = 0.0;
    int bcount = clamp(uBlobCount, 0, 8);
    for (int i = 0; i < bcount; i++) {
        if (uBlobScale[i] <= 0.001) continue;
        float dist = length(vWorld.xz - uBlobPos[i].xz);
        float rn = dist / (uShadowRadius * uBlobScale[i]);
        float tt = clamp((rn - uShadowInnerStop) / (1.0 - uShadowInnerStop), 0.0, 1.0);
        float a = tt < uShadowMidStop
            ? mix(uShadowInnerAlpha, uShadowMidAlpha, tt / uShadowMidStop)
            : mix(uShadowMidAlpha, 0.0, (tt - uShadowMidStop) / (1.0 - uShadowMidStop));
        max_shadow = max(max_shadow, a);
    }
    color = mix(color, lin2srgb(uShadowColor), max_shadow);

    // --- linear fog, Three smoothstep, display space ---
    float depth = -VERTEX.z;
    float fogF = smoothstep(uFogNear, uFogFar, depth);
    color = mix(color, lin2srgb(uFogColor), fogF);

    ALBEDO = srgb2lin(color);
}
```

FILE: shaders/prop.gdshader (59 lines)

```
shader_type spatial;
// Prop material: flat-shaded lit surface (the HTML used MeshStandardMaterial with flatShading) plus the noise dissolve
// with a glowing edge that plays while a prop is being digested.
render_mode cull_disabled;

group_uniforms Surface;
uniform vec3 uColor : source_color = vec3(0.8, 0.8, 0.8);
uniform float uRoughness : hint_range(0.0, 1.0, 0.01) = 0.55;
uniform float uMetallic : hint_range(0.0, 1.0, 0.01) = 0.05;
uniform bool uFlatShading = true;

group_uniforms Dissolve;
uniform vec3 uGlow : source_color = vec3(0.25, 1.0, 0.7);
uniform float uGlowStrength = 2.5;
uniform float uNoiseScale = 3.0;
uniform float uThresholdScale = 1.25;
uniform float uThresholdBias = 0.1;
uniform float uEdgeWidth = 0.1;

group_uniforms;

// Driven by Prop.gd while being eaten: 0 = intact, 1 = gone
uniform float uDissolve = 0.0;

varying vec3 vObjPos;

float dh(vec3 p) {
    return fract(sin(dot(p, vec3(127.1, 311.7, 74.7))) * 43758.5453);
}

float dvn(vec3 p) {
    vec3 i = floor(p);
    vec3 f = fract(p);
    f = f * f * (3.0 - 2.0 * f);
    return mix(
        mix(mix(dh(i), dh(i + vec3(1.0, 0.0, 0.0)), f.x), mix(dh(i + vec3(0.0, 1.0, 0.0)), dh(i + vec3(1.0, 1.0, 0.0)), f.x), f.y),
        mix(mix(dh(i + vec3(0.0, 0.0, 1.0)), dh(i + vec3(1.0, 0.0, 1.0)), f.x), mix(dh(i + vec3(0.0, 1.0, 1.0)), dh(i + vec3(1.0, 1.0, 1.0)), f.x), f.y),
        f.z);
}

void vertex() {
    vObjPos = VERTEX;
}

void fragment() {
    float dn = dvn(vObjPos * uNoiseScale);
    float dth = uDissolve * uThresholdScale - uThresholdBias;
    if (dn < dth) {
        discard;
    }
    ALBEDO = uColor;
    ROUGHNESS = uRoughness;
    METALLIC = uMetallic;
    if (uFlatShading) {
        NORMAL = normalize(cross(dFdx(VERTEX), dFdy(VERTEX)));
    }
    float dedge = 1.0 - smoothstep(0.0, uEdgeWidth, dn - dth);
    EMISSION = uGlow * dedge * uGlowStrength * step(0.001, uDissolve);
}
```

FILE: scenes/main.tscn (289 lines)

```
[gd_scene format=3 uid="uid://cebnw7hsbwt86"]

[ext_resource type="Shader" uid="uid://cdylotpifnx5w" path="res://shaders/slime.gdshader" id="1_slime_shader"]
[ext_resource type="Script" uid="uid://d0v3rofmqon45" path="res://scripts/slime.gd" id="3_slime_script"]
[ext_resource type="Script" uid="uid://be8ahyc03pcny" path="res://scripts/orbit_camera.gd" id="4_orbit_script"]
[ext_resource type="Shader" uid="uid://qeelh8bvw3or" path="res://shaders/floor.gdshader" id="5_floor_shader"]
[ext_resource type="Script" uid="uid://bsn3fclfde6cq" path="res://scripts/props.gd" id="6_props_script"]
[ext_resource type="Script" uid="uid://b1jvytjt3j1k4" path="res://scripts/hud.gd" id="7_hud_script"]

[sub_resource type="Environment" id="Environment_studio"]
background_mode = 1
background_color = Color(0.337, 0.471, 0.565, 1)
ambient_light_source = 2
ambient_light_color = Color(0.859, 0.918, 0.996, 1)
ambient_light_energy = 0.85
fog_enabled = true
fog_mode = 1
fog_light_color = Color(0.337, 0.471, 0.565, 1)
fog_density = 1.0
fog_depth_begin = 25.0
fog_depth_end = 90.0

[sub_resource type="ShaderMaterial" id="ShaderMaterial_slime"]
render_priority = 0
shader = ExtResource("1_slime_shader")
shader_parameter/uJiggleStrength = 1.0
shader_parameter/uDispersion = 0.85
shader_parameter/uRefractionStrength = 0.32
shader_parameter/uRippleNoiseScale = 1.65
shader_parameter/uBubbleNoiseScale = 3.5
shader_parameter/uRimStrength = 1.15
shader_parameter/uOpacity = 0.48
shader_parameter/uNightMode = 0.0
shader_parameter/uTopGlow = Vector3(0.08, 0.18, 0.28)
shader_parameter/uBubbleBoost = 1.4
shader_parameter/uDayShallow = Color(0.52, 0.82, 0.98, 1)
shader_parameter/uDayDeep = Color(0.28, 0.62, 0.9, 1)
shader_parameter/uNightShallow = Color(0.22, 0.64, 0.94, 1)
shader_parameter/uNightDeep = Color(0.08, 0.32, 0.64, 1)
shader_parameter/uRimDay = Color(0.8, 0.96, 1, 1)
shader_parameter/uRimNight = Color(0.62, 0.92, 1, 1)
shader_parameter/uOutlineDay = Color(0.16, 0.34, 0.52, 1)
shader_parameter/uOutlineNight = Color(0.04, 0.12, 0.24, 1)
shader_parameter/uCausticA = Color(0.18, 0.92, 0.62, 1)
shader_parameter/uCausticB = Color(0.42, 0.98, 0.35, 1)
shader_parameter/uBubbleColor = Color(0.85, 1, 0.95, 1)
shader_parameter/uEyeColor = Color(1, 1, 1, 1)
shader_parameter/uEyeSeparation = 0.32
shader_parameter/uEyeSize = Vector2(0.11, 0.17)
shader_parameter/uEyeElevation = 0.12
shader_parameter/uMelt = 0.28
shader_parameter/uImpulseTime = 10.0
shader_parameter/uImpulsePos = Vector3(0, 0.5, 1)
shader_parameter/uImpulseVector = Vector3(0, 0, 0)
shader_parameter/uImpulse2Time = 10.0
shader_parameter/uImpulse2Pos = Vector3(0, 0.5, 1)
shader_parameter/uImpulse2Vector = Vector3(0, 0, 0)
shader_parameter/uMoveDir = Vector3(0, 0, 1)
shader_parameter/uMoveAmt = 0.0
shader_parameter/uCrawl = 0.0
shader_parameter/uLean = Vector3(0, 0, 0)
shader_parameter/uEatPos = PackedVector3Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
shader_parameter/uEatParam = PackedVector3Array(0, 0, 0
```

```
, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
shader_parameter/uBoxHalf = PackedVector3Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
shader_parameter/uPressDir = Vector3(0, 0, 1)
shader_parameter/uPressAmt = 0.0
shader_parameter/uSplatDir = Vector3(0, 0, 1)
shader_parameter/uSplat = 0.0
shader_parameter/uSpread = 0.0
shader_parameter/uWall = 2.15
shader_parameter/uSolidPos = PackedVector3Array(0, 0, 0, 0, 0, 0)
shader_parameter/uSolidParam = PackedVector3Array(0, 0, 0, 0, 0, 0)
shader_parameter/uSolidHalf = PackedVector3Array(0, 0, 0, 0, 0, 0)
shader_parameter/uSolidDir = PackedVector3Array(0, 0, 0, 0, 0, 0)
shader_parameter/uRippleIntensity = 0.12
shader_parameter/uFacing = 0.45
shader_parameter/uBlink = 1.0
shader_parameter/uSqueeze = 0.0
shader_parameter/uSqueezeDir = Vector3(1, 0, 0)
shader_parameter/uLiquid = 0.0
shader_parameter/uDrapePos = Vector3(0, 0, 0)
shader_parameter/uDrapeHalf = Vector3(0.01, 0.01, 0.01)
shader_parameter/uDrapeShape = 0.0
shader_parameter/uDrapeYaw = 0.0
shader_parameter/uDrapeAmt = 0.0
shader_parameter/uSplitAmt = 0.0
shader_parameter/uSplitDir = Vector3(0, 0, -1)
shader_parameter/uBudAmt = 0.0
shader_parameter/uBudDir = Vector3(0, 0, -1)
shader_parameter/uBudScale = 0.0

[sub_resource type="SphereMesh" id="SphereMesh_slime"]
radius = 1.6
height = 3.2
radial_segments = 160
rings = 120

[sub_resource type="StandardMaterial3D" id="Mat_red"]
albedo_color = Color(0.867, 0.373, 0.373, 1)
roughness = 0.55

[sub_resource type="BoxMesh" id="BoxMesh_prop"]
size = Vector3(1.4, 1.4, 1.4)

[sub_resource type="StandardMaterial3D" id="Mat_orange"]
albedo_color = Color(0.867, 0.62, 0.373, 1)
roughness = 0.55

[sub_resource type="StandardMaterial3D" id="Mat_lime"]
albedo_color = Color(0.62, 0.867, 0.373, 1)
roughness = 0.55

[sub_resource type="StandardMaterial3D" id="Mat_teal"]
albedo_color = Color(0.373, 0.867, 0.7, 1)
roughness = 0.55

[sub_resource type="StandardMaterial3D" id="Mat_blue"]
albedo_color = Color(0.373, 0.56, 0.867, 1)
roughness = 0.55

[sub_resource type="StandardMaterial3D" id="Mat_purple"]
albedo_color = Color(0.7, 0.373, 0.867, 1)
roughness = 0.55

[sub_resource type="ShaderMaterial" id="ShaderMaterial_floor"]
render_priority = 0
shader = ExtResource("5_floor_shader")
shader_parameter/uDevTexture = false
shader_parameter/uCellSize = 1.0
shader_parameter/uMajorEvery = 4.0
shader_parameter/uLightCell = Color(0.46, 0.5, 0.54, 1)
shader_parameter/uDarkCell = Color(0.37, 0.41, 0.45, 1)
shader_parameter/uMinorLine = Color(0.3, 0.34, 0.38, 1)
shader_parameter/uMajorLine = Color(0.85, 0.91, 0.97, 1)
shader_parameter/uMinorWidth = 0.025
shader_parameter/uMajorWidth = 0.02
shader_parameter/uMinorAlpha = 0.6
shader_parameter/uMajorAlpha = 0.85
shader_parameter/uFadeStart = 0.35
shader_parameter/uFadeEnd = 0.8
shader_parameter/uTiles = 100.0
shader_parameter/uPlaneSize = 400.0
shader_parameter/uBaseColor = Color(0.435, 0.49, 0.549, 1)
shader_parameter/uEdgeColor = Color(0.824, 0.91, 0.98, 1)
shader_parameter/uEdgeAlpha = 0.55
shader_parameter/uMidAlpha = 0.18
shader_parameter/uEdgeHalfWidth = 0.00390625
shader_parameter/uMidHalfWidth = 0.001953125
shader_parameter/uAmbientColor = Color(0.859, 0.918, 0.996, 1)
shader_parameter/uAmbientEnergy = 0.85
shader_parameter/uLightColor = Color(1, 1, 1, 1)
shader_parameter/uLightEnergy = 0.9
shader_parameter/uLightDir = Vector3(4, 8, 5)
shader_parameter/uFogColor = Color(0.337, 0.471, 0.565, 1)
shader_parameter/uFogNear = 25.0
shader_parameter/uFogFar = 90.0
shader_parameter/uShadowColor = Color(0.039, 0.078, 0.137, 1)
shader_parameter/uShadowInnerAlpha = 0.6
shader_parameter/uShadowMidAlpha = 0.35
shader_parameter/uShadowMidStop = 0.7
shader_parameter/uShadowInnerStop = 0.0625
shader_parameter/uShadowRadius = 2.0
shader_parameter/uBlobCount = 1
shader_parameter/uBlobPos = PackedVector3Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
shader_parameter/uBlobScale = PackedFloat32Array(0, 0, 0, 0, 0, 0, 0, 0)

[sub_resource type="PlaneMesh" id="PlaneMesh_floor"]
size = Vector2(400, 400)

[node name="Main" type="Node3D" unique_id=1750818781]

[node name="WorldEnvironment" type="WorldEnvironment" parent="." unique_id=1848971336]
environment = SubResource("Environment_studio")

[node name="Sun" type="DirectionalLight3D" parent="." unique_id=13382104]
transform = Transform3D(0.7809, -0.4877, 0.3904, 0, 0.625, 0.7807, -0.6247, -0.6096, 0.488, 4, 8, 5)
light_energy = 0.9

[node name="Slime" type="Node3D" parent="." unique_id=1693722551 node_paths=PackedStringArray("body", "camera", "floor_mesh", "props")]
script = ExtResource("3_slime_script")
body = NodePath("Body")
camera = NodePath("../CameraRig/Camera3D")
floor_mesh = NodePath("../Floor")
props = NodePath("../Ecosystem")

[node name="Body" type="MeshInstance3D" parent="Slime" unique_id=512345633]
material_override = SubResource("ShaderMaterial_slime")
cast_shadow = 0
extra_cull_margin = 4.0
mesh = SubResource("SphereMesh_slime")

[node name="CameraRig" type="Node3D" parent="." unique_id=870445326 node_paths=PackedStringArray("camera", "follow")]
script = ExtResource("4_orbit_script")
camera = NodePath("Camera3D")
follow = NodePath("../Slime")

[node name="Camera3D" type="Camera3D" parent="CameraRig" unique_id=1187376853]
current = true
fov = 42.0
near = 0.1
far = 100.0

[node name="Props" type="Node3D" parent="." unique_id=432529286]
visible = false

[node name="Cube01" type="MeshInstance3D" parent="Props" unique_id=720179014]
transform = Transform3D(0.7518, 0, 0.2736, 0, 0.8, 0, -0.2736, 0, 0.7518, 6.58, -0.64, 2.39)
material_override = SubResource("Mat_red")
mesh = SubResource("BoxMesh_prop")

[node name="Cube02" type="MeshInstance3D" parent="Props" unique_id=894671729]
transform = Transform3D(0.5916, 0, 1.2688, 0, 1.4, 0, -1.2688, 0, 0.5916, 5.45, -0.22, 7.78)
material_override = SubResource("Mat_orange")
mesh = SubResource("BoxMesh_prop")

[node name="Cube03" type="MeshInstance3D" parent="Props" unique_id=1185253857]
transform = Transform3D(0.5909, 0, 0.1042, 0, 0.6, 0, -0.1042, 0, 0.5909, -0.57, -0.78, 6.48)
material_override = SubResource("Mat_lime")
mesh = SubResource("BoxMesh_prop")

[node name="Cube04" type="MeshInstance3D" parent="Props/Cube03" unique_id=1497054997]
transform = Transform3D(2.5978868, 0, 1.4999156, 0, 2.999998, 0, -1.4999156, 0, 2.5978868, -11.232807, 1.4, 1.3192453)
material_override = SubResource("Mat_teal")
mesh = SubResource("BoxMesh_prop")

[node name="Cube05" type="MeshInstance3D" parent="Props" unique_id=757536885]
transform = Transform3D(0.1736, 0, 0.9848, 0, 1, 0, -0.9848, 0, 0.1736, -7.73, -0.5, 2.07)
material_override = SubResource("Mat_blue")
mesh = SubResource("BoxMesh_prop")

[node name="Cube06" type="MeshInstance3D" parent="Props" unique_id=1714722357]
transform = Transform3D(1.9939, 0, 0.9297, 0, 2.2, 0, -0.9297, 0, 1.9939, -11.75, 0.34, -4.28)
material_override = SubResource("Mat_purple")
mesh = SubResource("BoxMesh_prop")

[node name="Cube07" type="MeshInstance3D" parent="Props" unique_id=67719004]
transform = Transform3D(0.4015, 0, 0.5734, 0, 0.7, 0, -0.5734, 0, 0.4015, -4.3, -0.71, -6.14)
material_override = SubResource("Mat_red")
mesh = SubResource("BoxMesh_prop")

[node name="Cube08" type="MeshInstance3D" parent="Props" unique_id=471657235]
transform = Transform3D(0.983, 0, 0.6883, 0, 1.2, 0, -0.6883, 0, 0.983, 0, -0.36, -10)
material_override = SubResource("Mat_lime")
mesh = SubResource("BoxMesh_prop")

[node name="Cube09" type="MeshInstance3D" parent="Props" unique_id=267745832]
transform = Transform3D(0.171, 0, 0.4699, 0, 0.5, 0, -0.4699, 0, 0.171, 3.9, -0.85, -5.57)
material_override = SubResource("Mat_teal")
mesh = SubResource("BoxMesh_prop")

[node name="Cube10" type="MeshInstance3D" parent="Props" unique_id=1109010432]
transform = Transform3D(1.5454, 0, 0.4141, 0, 1.6, 0, -0.4141, 0, 1.5454, 11.78, -0.08, -5.49)
material_override = SubResource("Mat_orange")
mesh = SubResource("BoxMesh_prop")

[node name="Cube11" type="MeshInstance3D" parent="Props" unique_id=757717056]
transform = Transform3D(1.8385, 0, 1.8385, 0, 2.6, 0, -1.8385, 0, 1.8385, 10.61, 0.62, 10.61)
material_override = SubResource("Mat_blue")
mesh = SubResource("BoxMesh_prop")

[node name="Cube12" type="MeshInstance3D" parent="Props" unique_id=237775911]
transform = Transform3D(0.7794, 0, 0.45, 0, 0.9, 0, -0.45, 0, 0.7794, -5.47, -0.57, -15.04)
material_override = SubResource("Mat_purple")
mesh = SubResource("BoxMesh_prop")


[node name="Floor" type="MeshInstance3D" parent="." unique_id=1904116251]
transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 0, -1.2, 0)
material_override = SubResource("ShaderMaterial_floor")
cast_shadow = 0
mesh = SubResource("PlaneMesh_floor")

[node name="Ecosystem" type="Node3D" parent="." unique_id=1530482977 node_paths=PackedStringArray("slime", "floor_mesh")]
script = ExtResource("6_props_script")
slime = NodePath("../Slime")
floor_mesh = NodePath("../Floor")

[node name="HudLayer" type="CanvasLayer" parent="." unique_id=1288801345]

[node name="Hud" type="Control" parent="HudLayer" unique_id=1740026618 node_paths=PackedStringArray("slime", "props")]
layout_mode = 3
anchors_preset = 15
anchor_right = 1.0
anchor_bottom = 1.0
grow_horizontal = 2
grow_vertical = 2
mouse_filter = 2
script = ExtResource("7_hud_script")
slime = NodePath("../../Slime")
props = NodePath("../../Ecosystem")
```

FILE: scenes/slime.tscn (69 lines)

```
[gd_scene format=3 uid="uid://drulaqw3tv8fk"]

[ext_resource type="Shader" uid="uid://cdylotpifnx5w" path="res://shaders/slime.gdshader" id="1_shader"]
[ext_resource type="Script" uid="uid://d0v3rofmqon45" path="res://scripts/slime.gd" id="2_script"]

[sub_resource type="ShaderMaterial" id="ShaderMaterial_instance"]
render_priority = 0
shader = ExtResource("1_shader")
shader_parameter/uJiggleStrength = 1.0
shader_parameter/uDispersion = 0.85
shader_parameter/uRefractionStrength = 0.32
shader_parameter/uRippleNoiseScale = 1.65
shader_parameter/uBubbleNoiseScale = 3.5
shader_parameter/uRimStrength = 1.15
shader_parameter/uOpacity = 0.48
shader_parameter/uNightMode = 0.0
shader_parameter/uTopGlow = Vector3(0.08, 0.18, 0.28)
shader_parameter/uBubbleBoost = 1.4
shader_parameter/uDayShallow = Color(0.52, 0.82, 0.98, 1)
shader_parameter/uDayDeep = Color(0.28, 0.62, 0.9, 1)
shader_parameter/uNightShallow = Color(0.22, 0.64, 0.94, 1)
shader_parameter/uNightDeep = Color(0.08, 0.32, 0.64, 1)
shader_parameter/uRimDay = Color(0.8, 0.96, 1, 1)
shader_parameter/uRimNight = Color(0.62, 0.92, 1, 1)
shader_parameter/uOutlineDay = Color(0.16, 0.34, 0.52, 1)
shader_parameter/uOutlineNight = Color(0.04, 0.12, 0.24, 1)
shader_parameter/uCausticA = Color(0.18, 0.92, 0.62, 1)
shader_parameter/uCausticB = Color(0.42, 0.98, 0.35, 1)
shader_parameter/uBubbleColor = Color(0.85, 1, 0.95, 1)
shader_parameter/uEyeColor = Color(1, 1, 1, 1)
shader_parameter/uEyeSeparation = 0.32
shader_parameter/uEyeSize = Vector2(0.11, 0.17)
shader_parameter/uEyeElevation = 0.12
shader_parameter/uMelt = 0.28
shader_parameter/uImpulseTime = 10.0
shader_parameter/uImpulsePos = Vector3(0, 0.5, 1)
shader_parameter/uImpulseVector = Vector3(0, 0, 0)
shader_parameter/uImpulse2Time = 10.0
shader_parameter/uImpulse2Pos = Vector3(0, 0.5, 1)
shader_parameter/uImpulse2Vector = Vector3(0, 0, 0)
shader_parameter/uMoveDir = Vector3(0, 0, 1)
shader_parameter/uMoveAmt = 0.0
shader_parameter/uCrawl = 0.0
shader_parameter/uLean = Vector3(0, 0, 0)
shader_parameter/uRippleIntensity = 0.12
shader_parameter/uFacing = 0.45
shader_parameter/uBlink = 1.0
shader_parameter/uBudAmt = 0.0
shader_parameter/uBudDir = Vector3(0, 0, -1)
shader_parameter/uBudScale = 0.0
shader_parameter/uSplitAmt = 0.0
shader_parameter/uSplitDir = Vector3(0, 0, -1)

[sub_resource type="SphereMesh" id="SphereMesh_body"]
radius = 1.6
height = 3.2
radial_segments = 120
rings = 90

[node name="Slime" type="Node3D" node_paths=PackedStringArray("body")]
script = ExtResource("2_script")
body = NodePath("Body")

[node name="Body" type="MeshInstance3D" parent="."]
material_override = SubResource("ShaderMaterial_instance")
cast_shadow = 0
extra_cull_margin = 4.0
mesh = SubResource("SphereMesh_body")

```